



NATIONAL UNIVERSITY OF HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF ELECTRONICS AND TELECOMMUNICATIONS
DEPARTMENT OF COMPUTER AND EMBEDDED SYSTEMS



HOANG DUC HIEU - DO THANH DAT

BACHELOR'S THESIS:

**CNN OPTIMIZATION TECHNIQUES AND
IMPLEMENTATION ON NVIDIA JETSON NANO**

MAJOR: COMPUTER AND EMBEDDED SYSTEMS

Ho Chi Minh City, August 2026





NATIONAL UNIVERSITY OF HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF ELECTRONICS AND TELECOMMUNICATIONS
DEPARTMENT OF COMPUTER AND EMBEDDED SYSTEMS



HOANG DUC HIEU - DO THANH DAT
22207026 - 22207016

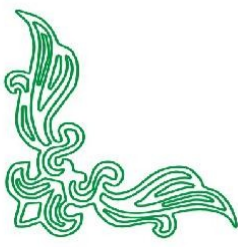
TITLE:

**CNN OPTIMIZATION TECHNIQUES AND
IMPLEMENTATION ON NVIDIA JETSON NANO**

BACHELOR'S THESIS
ELECTRONICS AND TELECOMMUNICATIONS ENGINEERING
COMPUTER AND EMBEDDED SYSTEMS

ACADEMIC SUPERVISORS
Dr. HUỖNH HỮU THUẬN
BSc. NGUYỄN MINH QUANG

Ho Chi Minh City, August 2026



Acknowledgements

The student group would like to express their sincere and deepest gratitude to the School Board, the Faculty Management, and all the lecturers of the Faculty of Electronics and Telecommunications at the University of Science, Vietnam National University - Ho Chi Minh City, for their dedicated teaching, facilitation, and support throughout our study and research at the university.

In particular, we would like to express our most sincere thanks to Dr. Huynh Huu Thuan and CN. Nguyen Minh Quang, our advisors who dedicated substantial time and enthusiasm to directly guide, orient our expertise, and support us in resolving difficulties during the project. Their valuable expert feedback helped us refine our research methods, enhance scientific rigor, ensure reproducibility, and cultivate a cautious empirical mindset.

We would also like to express our gratitude to our families, friends, colleagues, and classmates who have always been by our side, encouraging us, sharing our challenges, and supporting us both materially and spiritually. Although we have made great efforts to review and complete this thesis, it is inevitably bound to contain unintended shortcomings. We highly appreciate any feedback and guidance from the members of the Thesis Defense Committee to further improve this research.

DECLARATION

We hereby declare that the data and research results presented in this bachelor's thesis are truthful and accurate. We further declare that no content or results from other research works have been copied. All references and materials used in this thesis have been fully and clearly cited, with their original sources acknowledged in accordance with applicable regulations.

We accept full responsibility for any violation of the above declarations.

Signature

Signature

Do Thanh Dat

Hoang Duc Hieu

ABSTRACT

Iris recognition is a biometric verification technique that uses the distinctive texture of the iris. However, practical systems must handle image degradation, localization errors, occlusion, and the limited computational resources of edge devices. This thesis presents an end-to-end framework for open-set iris verification and its deployment on the NVIDIA Jetson Nano.

The proposed system localizes and normalizes the iris region before extracting discriminative embeddings using a deep neural network trained with ArcFace. A soft-mask preprocessing method is introduced to reduce the effects of eyelids, eyelashes, and invalid iris regions. The framework is evaluated using a subject-disjoint subset of the CASIA-Iris-Thousand dataset containing 1,000 subjects and 20,000 images.

Experimental results show that the proposed method achieves an equal error rate (EER) of 3.5498%, an area under the ROC curve (AUC) of 0.9921, and a true acceptance rate (TAR) of 88.1940% at a false acceptance rate (FAR) of 0.1%. It outperforms the baseline, which achieves an EER of 7.3549% and a TAR of 78.8451% at the same FAR.

For edge deployment, the trained model is exported to ONNX and optimized using TensorRT. On the Jetson Nano, TensorRT FP16 inference reaches 13.035 FPS, providing a $2.21\times$ speedup over PyTorch FP32 while maintaining nearly identical embedding outputs. These results demonstrate that the proposed framework improves verification accuracy while enabling efficient iris recognition on resource-constrained edge hardware.

CONTENTS

CHAPTER 1. INTRODUCTION.....	
1.1 Background and Motivation.....	
1.2 Global Applications of Iris Recognition.....	
1.3 Limitations of Existing Approaches.....	
1.4 Direction of This Thesis.....	
1.5 Research Objectives.....	
1.6 Research Scope.....	
1.7 Overall Methodology.....	
1.8 Achieved Results.....	
1.9 Contributions.....	
1.10 Thesis Organization.....	
CHAPTER 2. THEORETICAL FOUNDATIONS.....	
2.1 Overview of Iris Recognition.....	
2.2 Digital Image Processing in Iris Recognition.....	
2.3 Pupil and Iris Localization.....	
2.4 Circular Hough Transform (HoughCircles).....	
2.5 Daugman's Rubber Sheet Model.....	
2.6 Deep Learning and Convolutional Neural Networks.....	
2.7 ArcFace.....	
2.8 IResNet Backbone.....	
2.9 ArcIris.....	
2.10 Multi-Scale Feature Fusion (MSFF).....	
2.11 Grad-CAM++ Visualization.....	
2.12 Software Frameworks and Reproducibility.....	
2.13 TensorRT.....	
2.14 Jetson Nano and Edge AI.....	
2.15 Evaluation Metrics.....	
2.16 Related Work Synthesis, Research Gap, and Chapter Summary.....	
CHAPTER 3. SYSTEM ARCHITECTURE AND TECHNOLOGY STACK.....	
3.1 System Architecture Overview.....	
3.2 Software Architecture.....	
3.3 Technology Stack.....	
3.4 System Data Flow.....	
3.5 Training Architecture.....	
3.6 Inference and Jetson Nano Deployment Architecture.....	
3.7 Functional Requirements.....	
3.8 Non-functional Requirements.....	
3.9 Chapter Summary.....	

CHAPTER 4. METHODOLOGY AND IMPLEMENTATION.....	
4.1 Implementation Methodology Overview.....	
4.2 Dataset.....	
4.3 Data Preprocessing.....	
4.4 Pupil and Iris Localization.....	
4.5 Daugman Normalization and Subject-Disjoint Splitting.....	
4.6 Model Design.....	
4.7 Training Configuration.....	
4.8 Training Procedure.....	
4.9 Verification Evaluation Procedure.....	
4.10 Grad-CAM++ Visualization.....	
4.11 Model Export.....	
4.12 TensorRT Optimization.....	
4.13 Jetson Nano Deployment Procedure.....	
4.14 Chapter Summary.....	
CHAPTER 5. EXPERIMENTAL RESULTS AND EVALUATION.....	
5.1 Experimental Environment.....	
5.2 Dataset Accounting.....	
5.3 Iris Localization and Normalization Results.....	
5.4 Training Results.....	
5.5 Frozen-Test Verification Results.....	
5.6 Why a Closed-Set Confusion Matrix Is Not Used.....	
5.7 Model Comparison and Discussion.....	
5.8 Controlled Component Comparisons.....	
5.9 Qualitative Visualizations.....	
5.10 TensorRT and Jetson Nano Deployment Results.....	
5.11 Threats to Validity.....	
CHAPTER 6. CONCLUSION AND FUTURE WORK.....	
6.1 Main Results.....	
6.2 Contributions and Learning Outcomes.....	
6.2.1 Technical and Research Practice.....	
6.3 Limitations.....	
6.4 Future Work.....	
REFERENCES.....	

LIST OF ABBREVIATION

Abbreviation	Full Name	Description
AMP	Automatic Mixed Precision	A training technique that combines numerical precisions to reduce memory usage and computational cost.
AUC	Area Under the ROC Curve	The area under the ROC curve, summarizing the model's discrimination performance across different thresholds.
BN	Batch Normalization	A technique that normalizes intermediate activations within a neural network.
CNN	Convolutional Neural Network	A neural network architecture commonly used to extract visual features from images.
CPU	Central Processing Unit	The primary processor responsible for general-purpose computation.
CSV	Comma-Separated Values	A plain-text tabular format used for data splitting and logging.
CUDA	Compute Unified Device Architecture	NVIDIA's platform for GPU-accelerated parallel computing.
cuDNN	CUDA Deep Neural Network Library	A GPU-accelerated library for deep-learning operations built on CUDA.
EER	Equal Error Rate	The error rate at the operating point where FAR and FRR are approximately equal.
FAR	False Acceptance Rate	The proportion of impostor comparisons incorrectly accepted as genuine.
FP16	16-bit Floating Point	A 16-bit floating-point numerical representation.
FP32	32-bit Floating Point	A 32-bit floating-point numerical representation.
FPS	Frames Per Second	The number of images or video frames processed per second.
FRR	False Rejection Rate	The proportion of genuine comparisons incorrectly rejected.
GPU	Graphics Processing Unit	A processor designed to accelerate parallel computation.
IDO	Integro-Differential Operator	An operator used to estimate the boundaries of the pupil and iris.
INT8	8-bit Integer	An 8-bit integer representation commonly used for quantized inference.
JSON	JavaScript Object Notation	A structured text format used to store configuration data and results.

MSFF	Multi-Scale Feature Fusion	A method that combines features extracted at multiple spatial scales.
NIR	Near-Infrared	Near-infrared radiation used for iris image acquisition.
NPZ	NumPy Zipped Archive	A compressed file format used to store NumPy arrays and feature vectors.
ONNX	Open Neural Network Exchange	An open format for exchanging machine-learning models between different frameworks.
RAM	Random-Access Memory	The system's temporary memory used to store data and instructions during processing.
ROC	Receiver Operating Characteristic	A curve that plots the TAR against the FAR as the decision threshold changes.
SDK	Software Development Kit	A collection of software tools used for application development.
TAR	True Acceptance Rate	The proportion of genuine comparisons correctly
TensorRT	NVIDIA TensorRT	An NVIDIA software development kit for optimizing and accelerating deep-learning inference.
t-SNE	t-Distributed Stochastic Neighbor Embedding	A dimensionality-reduction method used to visualize high-dimensional data in a lower-dimensional space.
YAML	YAML Ain't Markup Language	A human-readable text format used to configure experiments and software applications.

Terminology convention: embedding (feature-vector representation), backbone (feature-extraction network), open-set verification (an evaluation protocol involving subjects unseen during training), checkpoint (saved model weights), split (a partition of dataset subjects), score (a similarity measure between two embeddings), and experimental artifact (any file or result generated during experiments).

LIST OF FIGURES

Figure 1.1. Visible iris texture and the pupil–iris–sclera structure used in biometric recognition.....	
Figure 1.2. Overall organization of the thesis.....	
Figure 2.1. Anatomical structure of the eye relevant to iris recognition.....	
Figure 2.2. Example of pupil and outer iris boundary localization.....	
Figure 2.3. Normalization of the iris region into polar coordinates using Daugman's rubber sheet model.....	
Figure 2.4. Geometric interpretation of the ArcFace angular margin [13].....	
Figure 2.5. Comparison of residual network architectures [12], [31].....	
Figure 2.6. NVIDIA Jetson Nano Developer Kit B01 used as the edge-inference platform.	
Figure 3.1. Schematic overview of the iris recognition system pipeline.....	
Figure 3.2. Detailed schematic of the training and checkpoint selection architecture.....	
Figure 3.3. Jetson Nano live recognition flow from camera acquisition to the browser-reported verification decision.....	
Figure 3.4. Hardware configuration for live iris-image acquisition and edge inference.....	
Figure 4.1. Representative preprocessing stages of the iris-recognition pipeline: (a) input grayscale eye image; (b) detected pupil boundary shown in yellow and outer iris boundary shown in green, together with localization metadata; (c) segmented iris annulus isolated from the surrounding eye region; and (d) normalized polar iris representation generated using Daugman's rubber-sheet model.....	
Figure 4.2. Preprocessing and localization preview from the experimental notebook.....	
Figure 4.3. Fresh static render of the IMX219 NoIR live-capture interface, showing raw and processed previews, quality guidance, and the three-step capture flow.....	
Figure 5.1. Validation EER by epoch for the final b1, b3, and MSFF neural runs; checkpoint selection used the minimum validation EER in each run.....	
Figure 5.2. Test ROC curve and matching score distribution from the prototype notebook run.....	
Figure 5.3. Qualitative t-SNE projection of the embedding space from the prototype notebook run.....	
Figure 5.4. Grad-CAM++ activation heatmaps and inverse projections from the prototype notebook run.....	
Figure 5.5. Real TensorRT FP16 recognition result produced by the deployed Jetson Nano WebUI using the final checkpoint.....	
Figure 5.6. Successful enrollment of a pseudonymous user using left- and right-eye iris embeddings.....	
Figure 5.7: Inference throughput on the Jetson Nano at Batch 1.....	

LIST OF TABLES

Table 3.1. Primary software modules and functional responsibilities.....	
Table 3.2. System technology stack and verification status.....	
Table 4.1. Pupil detection parameters using the circle Hough transform.....	
Table 4.3. Iris normalization parameters using Daugman's model.....	
Table 4.4. Dataset partition statistics.....	
Table 4.5. Subject-disjoint dataset splitting configuration.....	
Table 4.6. Architectural configurations and parameter counts of the evaluated models.....	
Table 4.7. Hyperparameter configuration for model training.....	
Table 4.8. ONNX input and output specifications.....	
Table 5.1. Experimental environments and verification status.....	
Table 5.2. Dataset partitioning and statistics.....	
Table 5.3. Preprocessing statistics and retention rate.....	
Table 5.4. Quantitative verification performance on the frozen CASIA test protocol.....	
Table 5.5. Provenance and status of the results used in the final comparison.....	
Table 5.6. Comparative analysis of baseline, proposed, and classical models.....	
Table 5.7. Status of controlled architectural comparisons and component analyses.....	
Table 5.8. Performance benchmarks and precision comparison on Jetson Nano.....	

CHAPTER 1. INTRODUCTION

1.1 Background and Motivation

Modern authentication systems use biometric traits to complement knowledge- or possession-based credentials. Unlike passwords, cards, or personal identification numbers, biometric samples are linked to a person; however, they can still be captured poorly, attacked, or mishandled and therefore do not remove the need for secure system design. Common modalities include face, fingerprint, voice, gait, retina, and iris recognition [1]–[5].

The iris is the annular structure between the pupil and sclera. Its visible texture contains crypts, furrows, freckles, and other patterns that are highly discriminative under controlled acquisition and relatively stable after early development [1]–[5]. These properties support iris verification, but performance still depends on sensor characteristics, illumination, gaze, focus, occlusion, and segmentation quality.

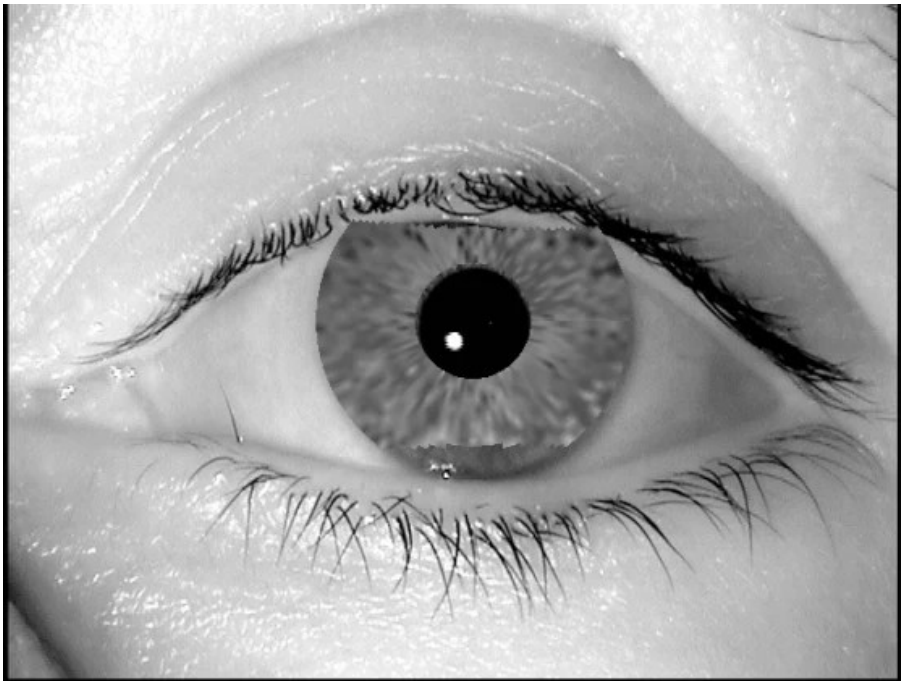


Figure 1.1. Visible iris texture and the pupil–iris–sclera structure used in biometric recognition.

However, the performance of an iris recognition system is not solely determined by the deep learning model. In practical scenarios, acquired ocular images are frequently degraded by motion blur, illumination variations, specular reflections, or occlusions from eyeglasses, eyelids, and eyelashes. Furthermore, inaccuracies in pupil and iris boundary localization present substantial challenges. Inaccurate segmentation or inconsistent normalization of the iris region can introduce significant noise, causing the deep learning

model to extract spurious peripheral features (such as eyelids or background regions) rather than the actual iris texture.

Additionally, practical deployment introduces substantial challenges because deep learning models are often computationally expensive and resource-intensive. In latency-sensitive authentication applications, transmitting biometric images to a cloud server increases system latency, depends heavily on network stability, and exposes sensitive biological data to privacy risks. Consequently, executing local inference on edge devices like the NVIDIA Jetson Nano provides significant practical utility, enabling data processing at the source of acquisition, optimizing transmission bandwidth, and supporting offline operations.

To address these requirements, this thesis focuses on constructing a baseline iris recognition framework that integrates classical image processing techniques for localization and normalization based on Daugman's rubber sheet model, applies deep learning for feature embedding extraction, and establishes an optimized inference pipeline for edge deployment.

1.2 Global Applications of Iris Recognition

Iris recognition has been investigated for access control, border processing, large-scale identity management, and supplementary identity verification. Its suitability depends on the acquisition environment, decision threshold, presentation-attack controls, and governance of biometric data; the modality should therefore be evaluated as one component of a broader security system [35]–[37].

On a broader scale, iris recognition serves as an effective tool for border control, passenger screening, and national identity management systems. In the banking and finance sector, this biometric modality supports secure transaction authentication, application login, and customer identification. When integrated into smart cameras and embedded systems, iris recognition enables localized security solutions at the edge. Nonetheless, practical deployment requires addressing challenges such as input image quality, spoofing vulnerabilities, processing speed, data privacy, and environmental robustness. Rather than building a complete commercial product, this thesis aims to lay the foundation for a reproducible pipeline and define optimization strategies on resource-constrained hardware.

1.3 Limitations of Existing Approaches

Traditional iris recognition methods rely heavily on the accuracy of image localization and normalization. When input images are affected by blur, poor illumination, specular reflections, or occlusions from eyelids and eyelashes, circle detection algorithms frequently fail to accurately locate the pupil or iris centers and radii. Even minor inaccuracies at this

stage can lead to misaligned normalized polar images, which degrades the quality of the extracted feature vector and directly reduces matching accuracy.

For deep learning models, the primary challenge lies in balancing verification accuracy with computational overhead. Large backbone architectures, such as IResNet50, offer powerful representation capabilities but possess complex structures that are difficult to deploy on resource-constrained devices without optimization. Conversely, lightweight models like MobileFaceNet are suitable for edge hardware but require thorough evaluation to ensure recognition stability.

Furthermore, many existing systems rely on cloud-based inference. While cloud computing provides substantial processing power, this paradigm is constrained by network bandwidth, latency, and a dependency on continuous connectivity, while also introducing security risks when transmitting raw biometric data. For highly sensitive data such as iris patterns, minimizing raw image transmission and prioritizing local processing at the source remains a critical requirement.

1.4 Direction of This Thesis

This thesis proposes a hybrid approach combining traditional image processing and deep learning. Specifically, the system localizes the pupil using the Circular Hough Transform (HoughCircles), determines the outer iris boundary using Daugman's integro-differential operator (IDO) with a Hough fallback mechanism when necessary, and normalizes the iris region into a fixed-size polar image of 64 x 512 dimensions using Daugman's rubber sheet model. This normalization step maps the annular iris structure into a uniform, flat representation prior to neural network processing.

During the deep learning phase, the feature extraction models are trained in conjunction with the ArcFace loss function to maximize inter-class separation and minimize intra-class distance in the feature space. The experimental configurations include: IResNet50 without a mask (b1), IResNet50 with a soft angular mask (b3), and the proposed IResNet50 integrated with a Multi-Scale Feature Fusion (MSFF) mechanism and a soft angular mask. The evaluations demonstrate that the baseline b1 configuration achieves an Equal Error Rate (EER) of 7.3549%, an Area Under the Curve (AUC) of 0.963291, and a True Acceptance Rate at a 0.1% False Acceptance Rate (TAR@0.1% FAR) of 78.8451%. The b3 configuration achieves an EER of 3.5915%, an AUC of 0.991927, and a TAR@0.1% FAR of 85.2568%. The proposed MSFF network yields an EER of 3.5498%, an AUC of 0.992076, and a TAR@0.1% FAR of 88.1940%. Thus, in this single-run comparison, the proposed configuration is numerically better than b1 on all three reported metrics: EER is lower by 3.8051 percentage points, AUC is higher by 0.028785, and TAR is higher by

9.3489 percentage points. It also reduces EER by 0.0417 percentage points relative to the matched b3 configuration. These differences are descriptive and are not claimed as statistically significant. For comparative context, the classical log-Gabor filter approach yields an EER of 8.9565%, an AUC of 0.959427, and a TAR@0.1% FAR of 75.2684%.

To prepare for edge deployment, the feature extraction embedding network is exported to the ONNX intermediate format, while pupil localization and normalization remain as host-side preprocessing operations. The deployment pipeline on the NVIDIA Jetson Nano platform has been fully completed and evaluated. Benchmarking results using the final checkpoint on the Jetson Nano show that the PyTorch FP32 baseline achieves a throughput of 5.920 FPS (168.660 ms latency), the TensorRT FP32 engine achieves 7.875 FPS (126.737 ms latency), and the TensorRT FP16 engine achieves 13.035 FPS (76.479 ms latency). The TensorRT FP16 engine is 2.21x faster than the PyTorch baseline, while maintaining high numerical consistency with a median cosine similarity of 0.999962. Biometric score consistency was verified through a conversion check on a 127-image deployment subset, though this does not represent the full CASIA validation protocol, and power consumption remains unreported. This edge deployment is subject to single-run execution and cross-sensor generalization limitations.

1.5 Research Objectives

The primary objective is to build and evaluate a reproducible iris-verification pipeline under one frozen subject-disjoint protocol, and to characterize the deployment behavior of its embedding network on a Jetson Nano without overstating the generality of a single-dataset study.

To achieve this primary objective, the following specific tasks are defined:

- Develop an ingestion pipeline for iris data adhering to the CASIA-Iris-Thousand dataset structure.
- Implement pupil and outer iris boundary localization algorithms for grayscale ocular images.
- Normalize the localized iris regions using Daugman's rubber sheet model.
- Train the feature extraction embedding models utilizing the ArcFace loss function.
- Evaluate model performance using an open-set verification protocol.
- Compare the performance of the completed baseline configurations.
- Generate analytical visualizations, including Receiver Operating Characteristic (ROC) curves, similarity score distributions, t-SNE projections, and Grad-CAM++ activation maps based on experimental evidence.
- Establish the workflow for exporting the trained embedding models to the ONNX format.

- Optimize the models via TensorRT and deploy them on the NVIDIA Jetson Nano edge platform.
- Accurately document any missing experimental results without fabricating or extrapolating data.

1.6 Research Scope

The scope of research encompasses human iris images, image preprocessing techniques (including pupil and outer iris boundary localization and geometric normalization), deep learning architectures for feature vector extraction, and verification evaluation procedures based on cosine similarity.

The scope of this study is bounded by the following limitations:

- The research focuses exclusively on iris recognition from static images and does not include the development of a commercial camera system.
- The experiments utilize the CASIA-Iris-Thousand dataset structure; the raw source dataset is not stored within the code repository.
- The primary evaluation protocol is open-set verification, rather than closed-set classification of the training classes.
- Edge deployment for local inference is successfully executed and benchmarked on the NVIDIA Jetson Nano platform.
- Liveness detection and anti-spoofing techniques are not implemented in the current codebase and are deferred to future work.
- The ONNX and TensorRT deployment pipeline is restricted to the feature extraction embedding network, whereas pupil localization and Daugman normalization are executed as host-side preprocessing operations.

1.7 Overall Methodology

The overall system workflow consists of sequential stages: ocular image acquisition, pupil and outer iris boundary localization, geometric normalization via Daugman's rubber sheet model, feature extraction embedding generation via a deep neural network, cosine similarity computation, and verification evaluation. The detailed diagram of this workflow is illustrated in Figure 3.1.

1.8 Achieved Results

Based on the empirical results obtained, this research has achieved the following outcomes:

- Refactored the initial codebase from experimental notebooks into modular, reusable Python components.
- Preserved the original experimental notebook `iris_baseline.ipynb` for reproducibility.

- Established a subject-exclusive dataset split comprising 700 subjects for training, 100 for validation, and 200 for testing.
- Verified that the dataset split contains 20,000 images across 1,000 subjects, representing 2,000 distinct eye classes with 10 images per class.
- Implemented the pupil and iris boundary localization pipeline alongside Daugman's rubber sheet normalization to produce 64 x 512 polar iris images.
- Implemented the baseline IResNet50 and the proposed MSFF configurations.
- Implemented the ArcFace loss head for deep embedding training.
- Established open-set verification evaluation using metrics such as Equal Error Rate (EER), Area Under the Curve (AUC), True Acceptance Rate at a 0.1% False Acceptance Rate (TAR@0.1% FAR), and False Acceptance/Rejection Rates.
- Generated visualization outputs in the source notebook including crop previews, training curves, ROC curves, similarity score distributions, t-SNE projections, and Grad-CAM++ heatmaps.
- Successfully exported the feature extraction embedding model to the ONNX format (opset 17) with verified computational graphs.
- The remaining open challenges and experimental constraints include:

The completed evidence covers three neural configurations, one evaluation-only classical comparison, ONNX export, and Jetson Nano measurements in PyTorch FP32, TensorRT FP32, and TensorRT FP16. The retired b4 attempt is excluded from the active comparison. Remaining limitations include one training run per neural configuration, unavailable input-power telemetry, incomplete run-specific localization statistics, and the absence of cross-sensor and presentation-attack evaluation.

1.9 Contributions

The primary scientific and practical contributions of this thesis include:

- Developing a fully reproducible, end-to-end iris recognition pipeline spanning data ingestion, preprocessing, training, and evaluation.
- Integrating classical image processing for iris localization and normalization with deep learning architectures for feature embedding generation.
- Implementing a subject-exclusive dataset splitting methodology to eliminate identity leakage across the training, validation, and testing partitions.
- Evaluating performance using a rigorous open-set verification protocol aligned with biometric standards.
- Developing a deployment-ready embedding model using ONNX and optimizing it with TensorRT for local inference on NVIDIA Jetson Nano edge hardware.
- Providing traceable configurations, frozen split metadata, metric artifacts, and an explicit separation between final measurements, historical visualizations, and unresolved limitations.

1.10 Thesis Organization

The remainder of this thesis is structured into six primary chapters:

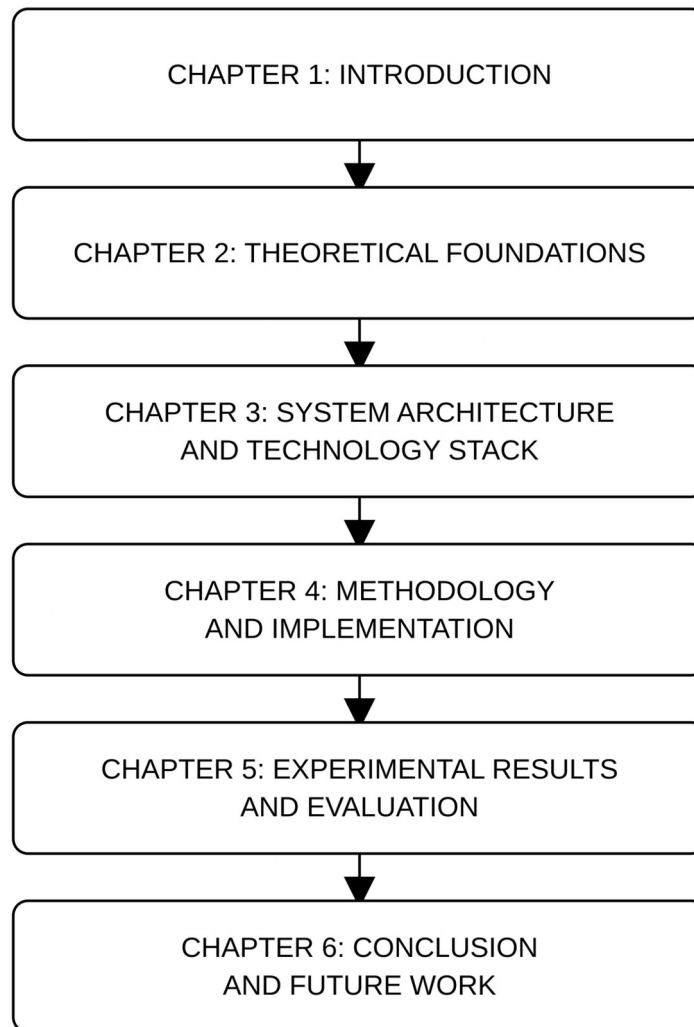


Figure 1.2. Overall organization of the thesis.

- Chapter 1 introduces the background, research objectives, scope, and contributions of this study.
- Chapter 2 reviews the theoretical foundations of iris recognition, image processing, deep learning backbones, and evaluation metrics.
- Chapter 3 describes the system architecture and the technology stack.
- Chapter 4 details the implementation methodology and techniques based on the actual codebase.
- Chapter 5 presents the empirical results and performance analysis of both the deep learning models and edge deployment.
- Chapter 6 concludes the thesis, discusses limitations, and outlines future research.

CHAPTER 2. THEORETICAL FOUNDATIONS

2.1 Overview of Iris Recognition

Iris recognition uses visible texture in the annular iris region for verification or identification. The iris contains crypts, contraction furrows, freckles, and other structures that can be highly discriminative under controlled imaging and are relatively stable after early development [1]–[5]. These properties support biometric use, but they do not make acquisition or matching error-free.

A standard iris recognition pipeline consists of eye image acquisition, segmentation, pupil and iris boundary localization, noise removal, region normalization, feature extraction, and matching [5]. Traditional methods typically encode features into a binary iris code [3], whereas modern deep learning architectures represent features as high-dimensional embeddings [10].

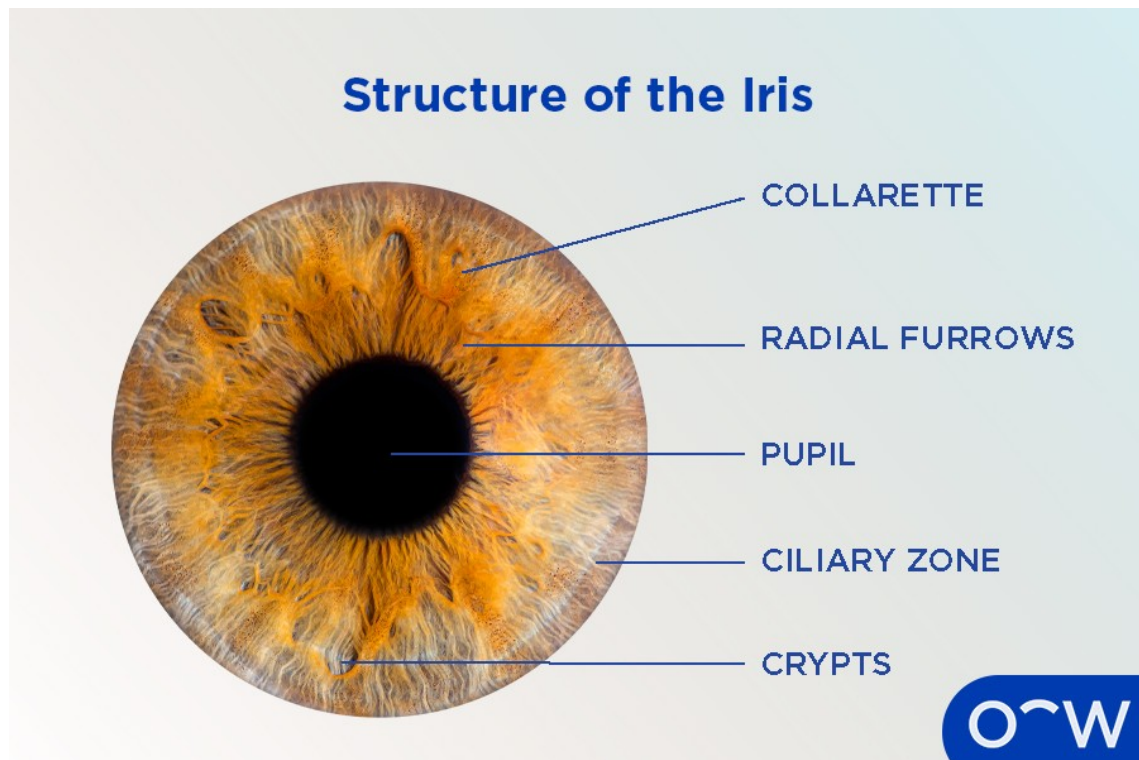


Figure 2.1. Anatomical structure of the eye relevant to iris recognition.

In biometrics, it is essential to distinguish among three concepts: classification, which assigns a class label during training; identification, which searches for the best-matching identity of an input sample in a database (one-to-many matching); and verification, which determines whether two image samples belong to the same individual (one-to-one matching) [22]. This study focuses on the open-set verification problem, where the subjects in the validation and testing sets do not overlap with the training set, reflecting real-world deployment scenarios.

Iris recognition exhibits characteristics such as uniqueness, stability over time, and resistance to spoofing [5]. However, the method faces challenges related to input image quality, acquisition distance, illumination variations, corneal reflection noise, occlusion from eyelids or eyelashes, localization errors, and pupil dilation [11]. In this system implementation, feature vectors are extracted from normalized iris images and compared using the cosine similarity metric.

2.2 Digital Image Processing in Iris Recognition

Digital image processing converts acquired eye images into a representation suitable for localization and feature extraction. A grayscale image stores one intensity value per pixel, reducing input dimensionality while retaining the boundary contrast used by many localization methods. Acquisition quality still varies with focus, illumination, gaze angle, pupil dilation, reflections, and occlusion [5], [36], [37].

Typical operations include resizing, denoising, contrast adjustment, intensity normalization, tensor conversion, and augmentation. These operations standardize the input but must be applied conservatively: excessive smoothing can remove fine iris texture, whereas insufficient processing can retain eyelid, eyelash, scleral, or reflection artifacts [5].

For this implementation, eye images are loaded in grayscale using OpenCV and smoothed with a median blur filter to reduce noise prior to circular Hough detection. The localized iris region is normalized using Daugman's rubber sheet model [3]. The normalized image is then converted to a PyTorch tensor, normalized using channel-wise mean and standard deviation, and augmented during training using spatial rolling, brightness and contrast jittering, random erasing, and soft angular masking.

2.3 Pupil and Iris Localization

Iris localization estimates the pupil boundary and the outer limbus so that the intervening annulus can be normalized. Classical systems use circular or integro-differential boundary models, whereas later systems also employ learned segmentation and occlusion estimation [2], [4]–[6], [16], [34]. Errors at this stage geometrically distort the normalized strip and can introduce non-iris pixels into matching.

Accurate localization isolates the region of interest and eliminates redundant background information. This is important for deep learning models, as incorrect cropping forces the model to process irrelevant noise features [10]. Localization failures introduce geometric distortions, include non-iris regions such as the sclera or eyelids, or exclude critical texture details.

In practice, pupil and iris boundaries are rarely perfectly circular and are often occluded by eyelids, eyelashes, eyeglasses, and specular reflections [34]. Additionally, pupil dilation changes the shape of the iris annulus. In this implementation, the circular Hough transform is applied for pupil detection, combined with an integro-differential operator (IDO) search and a Hough fallback mechanism for the outer iris boundary, followed by geometric constraint validation.

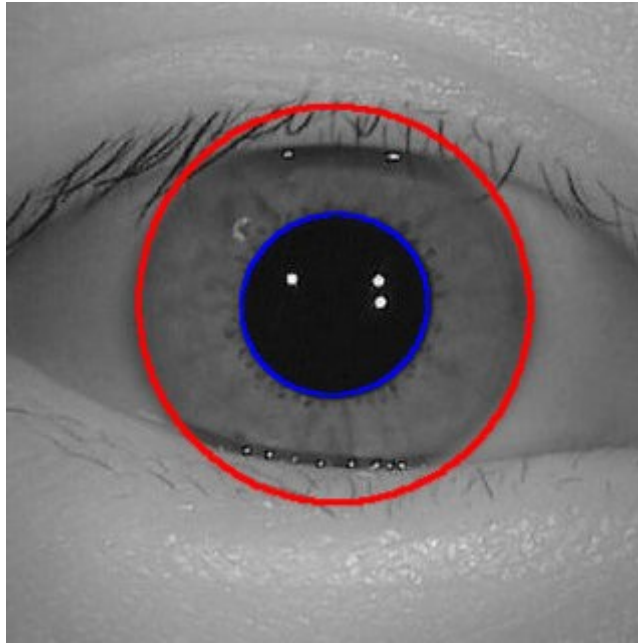


Figure 2.2. Example of pupil and outer iris boundary localization.

2.4 Circular Hough Transform (HoughCircles)

The circular Hough transform is a technique used to detect circles in an image by mapping edge pixels to a parameter space [32]. A circle in a two-dimensional coordinate system is defined by its center coordinates (x, y) and radius r . The transform maps the image space to a three-dimensional parameter space, where local extrema (accumulator peaks) indicate the presence of circular boundaries.

In the OpenCV library implementation, the `HOUGH_GRADIENT` method leverages edge gradient direction to constrain the search space. The operational parameters include the accumulator resolution ratio (`dp`), the minimum distance between circle centers (`minDist`), the upper Canny edge detection threshold (`param1`), the accumulator threshold (`param2`), and the search radius bounds (`minRadius` and `maxRadius`).

The circular Hough transform is training-free and useful when the target boundary is approximately circular [32]. Its estimates are nevertheless sensitive to noise, illumination variation, specular reflection, partial occlusion, and parameter ranges. For this reason, practical iris pipelines commonly combine geometric constraints, alternative boundary

estimators, or learned segmentation rather than treating a Hough peak as sufficient evidence [5], [16].

2.5 Daugman's Rubber Sheet Model

Daugman's rubber sheet model normalizes the iris region by mapping the annular region between the pupil boundary and the outer iris boundary to a polar coordinate system [3]. For each angular coordinate θ , the model determines a reference point on the pupil boundary and a corresponding point on the outer iris boundary. Intermediate pixels along the radial direction are linearly interpolated. Iterating over the angular range $[0, 2\pi]$ and the normalized radial interval maps the circular iris region into a rectangular polar representation.

General representation:

$$x(r, \theta) = (1 - r) x_p(\theta) + r x_i(\theta) \quad (2.1)$$

$$y(r, \theta) = (1 - r) y_p(\theta) + r y_i(\theta) \quad (2.2)$$

Where $x(r, \theta)$ and $y(r, \theta)$ are the Cartesian coordinates in the original image, $x_p(\theta)$ and $y_p(\theta)$ represent the pupil boundary coordinates, $x_i(\theta)$ and $y_i(\theta)$ represent the outer iris boundary coordinates, r is the normalized radial coordinate in the range $[0, 1]$, and θ is the angular coordinate in the range $[0, 2\pi]$.

This normalization generates a fixed-size representation, mitigating variations caused by pupil dilation or contraction and facilitating feature extraction. However, the quality of the normalized polar image depends on the accuracy of the localization stage. Incorrect localization introduces geometric distortions and includes non-iris regions such as the sclera, eyelids, or eyelashes.

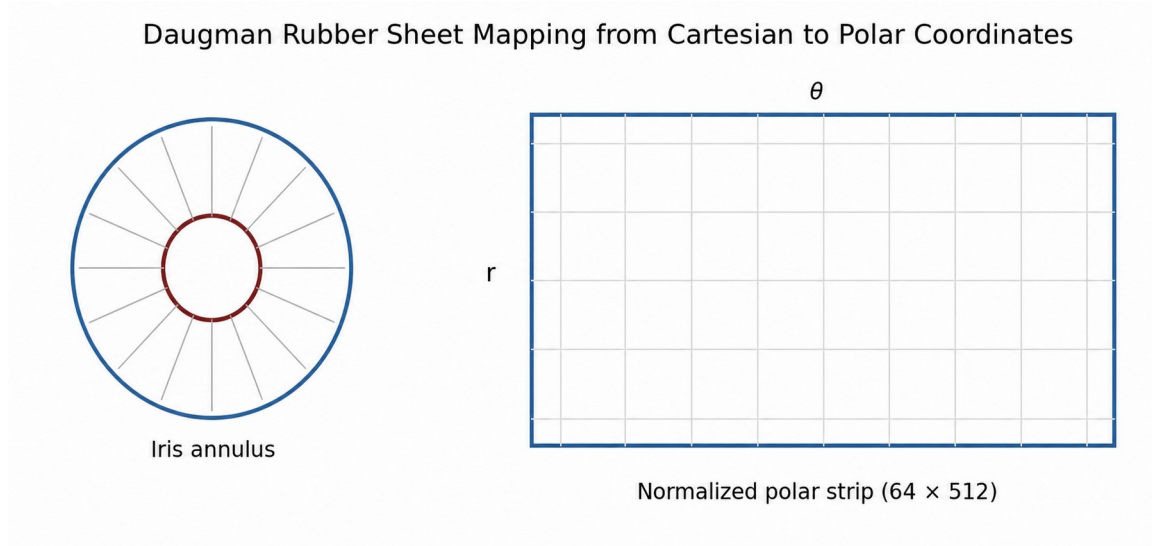


Figure 2.3. Normalization of the iris region into polar coordinates using Daugman's rubber sheet model.

2.6 Deep Learning and Convolutional Neural Networks

Convolutional neural networks learn hierarchical image representations through local connectivity and shared weights [31]. In iris recognition, deep models have been trained on normalized polar strips, raw or cropped ocular images, and spatially corresponding representations; these choices create different dependencies on segmentation accuracy and sensor domain [8]–[11], [29], [30].

Learned representations can reduce reliance on handcrafted descriptors, but they can also overfit sensor characteristics or identities if dataset partitions are not subject-disjoint. Comparisons therefore require a frozen protocol, clear checkpoint selection, and explicit reporting of acquisition and filtering conditions [11], [22], [35].

2.7 ArcFace

The Additive Angular Margin Loss (ArcFace) learns embeddings on a hypersphere by applying an angular margin to the ground-truth class [13]. For training sample i and class j , the feature vector and class-weight vector are L2-normalized as

$$\hat{x}_i = \frac{x_i}{\|x_i\|_2}, \quad \hat{w}_j = \frac{w_j}{\|w_j\|_2} \quad (2.3)$$

The angle between the normalized feature and class-weight vectors is defined by

$$\cos(\theta_j) = \hat{w}_j^T \hat{x}_i \quad (2.4)$$

For the ground-truth label, ArcFace adds angular margin m before applying the positive scale s . The scaled logits are

$$z_{i, y_i} = s \cos(\theta_{y_i} + m), \quad z_{i, j} = s \cos(\theta_j) \quad (j \neq y_i) \quad (2.5)$$

and the per-sample cross-entropy loss is

$$L_i = -\log \frac{\exp(z_{i, y_i})}{\sum_{j=1}^C \exp(z_{i, j})} \quad (2.6)$$

Here, C is the number of training classes, s is a positive scale, and m is the angular margin.

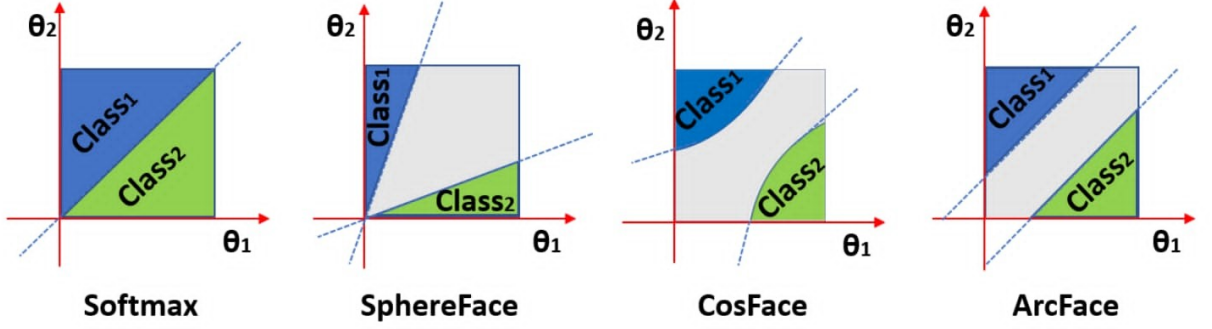


Figure 2.4. Geometric interpretation of the ArcFace angular margin [13]

The angular margin encourages embeddings of the same training class to occupy a compact angular region while separating class directions [13]. In an iris application, its benefit remains empirical: the margin, scale, data quality, and number of subject-eye classes affect optimization, and performance on unseen identities must be assessed with a subject-disjoint verification protocol [14].

2.8 IResNet Backbone

The IResNet backbone used in this study follows the residual-learning family, in which identity shortcuts improve gradient propagation through deep networks [12], [31]. The implementation is derived from the official InsightFace ArcFace codebase [40]. The name IResNet is retained as the implementation's model identifier rather than expanded into an unsupported product name.

Comparison Criterion	ResNet-34	ResNet-50	ResNet-101	ResNet-152
Deeper	No	No	Yes	Yes
Number of Parameters	21.8M	23.5M	42.7M	58.3M
Computational Complexity	Low	Moderate	High	Highest
Representation Power	Lower	Moderate	High	Highest
Feature Extraction Capability	Limited	Moderate	High	Highest
Improved Performance	–	Slightly Improved	Improved	Improved
Training Time	Shorter	Longer	Longer	Longer

Figure 2.5. Comparison of residual network architectures [12], [31].

Residual backbones offer substantial representational capacity but impose memory and latency costs. Lightweight or task-specific iris networks can reduce that cost, although the trade-off must be measured under the same data and verification protocol rather than inferred from parameter count alone [16], [20].

2.9 ArcIris

In this thesis, ArcIris is an internal configuration name for the combination of Daugman-normalized polar input, an embedding backbone, and ArcFace training. It is not introduced as a general theoretical method; its exact dimensions, masking options, and training configuration are specified in Chapter 4.

2.10 Multi-Scale Feature Fusion (MSFF)

Multi-Scale Feature Fusion (MSFF) aggregates feature maps from different network depths to preserve both fine spatial details and high-level semantic context [23]. In iris recognition, textural features range from local patterns to global structures. Combining intermediate feature maps, which retain high-resolution spatial boundaries, with deeper maps containing abstract representations, allows the network to leverage multi-scale features simultaneously.

Fusion requires spatial alignment and channel projection before features from different stages can be combined. It may recover fine texture that is attenuated in deeper layers, but it also increases parameters and can propagate localization artifacts. The exact layer selection and projection used in this project are implementation details reported in Chapter 4, and their effect is evaluated through the matched soft-mask comparison in Chapter 5.

2.11 Grad-CAM++ Visualization

Grad-CAM++ is a visualization method that provides visual explanations of convolutional neural network decisions [17]. It computes a weighted combination of the positive partial derivatives of feature maps in a target convolutional layer with respect to a specific class score to produce a coarse localization map. This heatmap is resized to match the input dimensions and overlaid to visualize the regional contributions to the network prediction.

For iris models, Grad-CAM++ can indicate whether a score is associated with regions inside or outside the normalized iris texture. Such heatmaps are qualitative explanations: they can reveal suspicious attention patterns but cannot verify robustness, fairness, or biometric accuracy [17].

When a heatmap is computed on a normalized polar strip, the localization parameters can be used to project it back to the Cartesian eye image. This projection aids visual interpretation but inherits any error in the estimated pupil and iris boundaries. The selected target layer and projection procedure are described in Chapter 4.

2.12 Software Frameworks and Reproducibility

Deep-learning frameworks provide automatic differentiation, accelerator-aware tensor operations, and standardized model serialization. Their scientific value in this thesis is reproducibility: the framework, preprocessing configuration, random seed, checkpoint rule, and evaluation code must be recorded together. Project-specific software responsibilities are presented in Chapter 3.

Model interchange formats separate a trained computational graph from a particular training program. ONNX provides the intermediate graph used in this work [26], while the target runtime and compatible CUDA, cuDNN, and JetPack components remain deployment-environment concerns described in Chapters 3 and 4 [38].

2.13 TensorRT

TensorRT is a high-performance deep learning inference optimizer and runtime library [24]. The deployment pipeline exports the trained model to the Open Neural Network Exchange (ONNX) format [26], which is then compiled into a hardware-specific engine on the target device. TensorRT optimizes execution through graph simplification, layer fusion, kernel selection, and precision calibration (e.g., FP16 and INT8) to minimize latency [39].

TensorRT accelerates inference by reducing latency and increasing throughput. On the Jetson Nano, half-precision (FP16) arithmetic optimizes execution for compatible layers [20]. However, the compilation pipeline requires addressing potential operator

incompatibilities, numerical deviations, or INT8 calibration complexities. Since the compiled engine is dependent on the specific hardware architecture and driver versions, compilation is performed directly on the target edge device.

A TensorRT engine is hardware- and software-stack specific, so it should be built on or for the target platform and checked numerically against a reference implementation. Latency, throughput, memory, and verification behavior must then be measured rather than estimated; this thesis reports those measurements in Chapter 5 [24], [39].

2.14 Jetson Nano and Edge AI

The Jetson Nano is an embedded system-on-module designed for local edge computing. Its architecture includes an NVIDIA Maxwell GPU with 128 CUDA cores, a quad-core ARM Cortex-A57 CPU, and 4GB of LPDDR4 memory [25]. The platform is used in research and prototyping for edge-deployed computer vision and biometric applications.

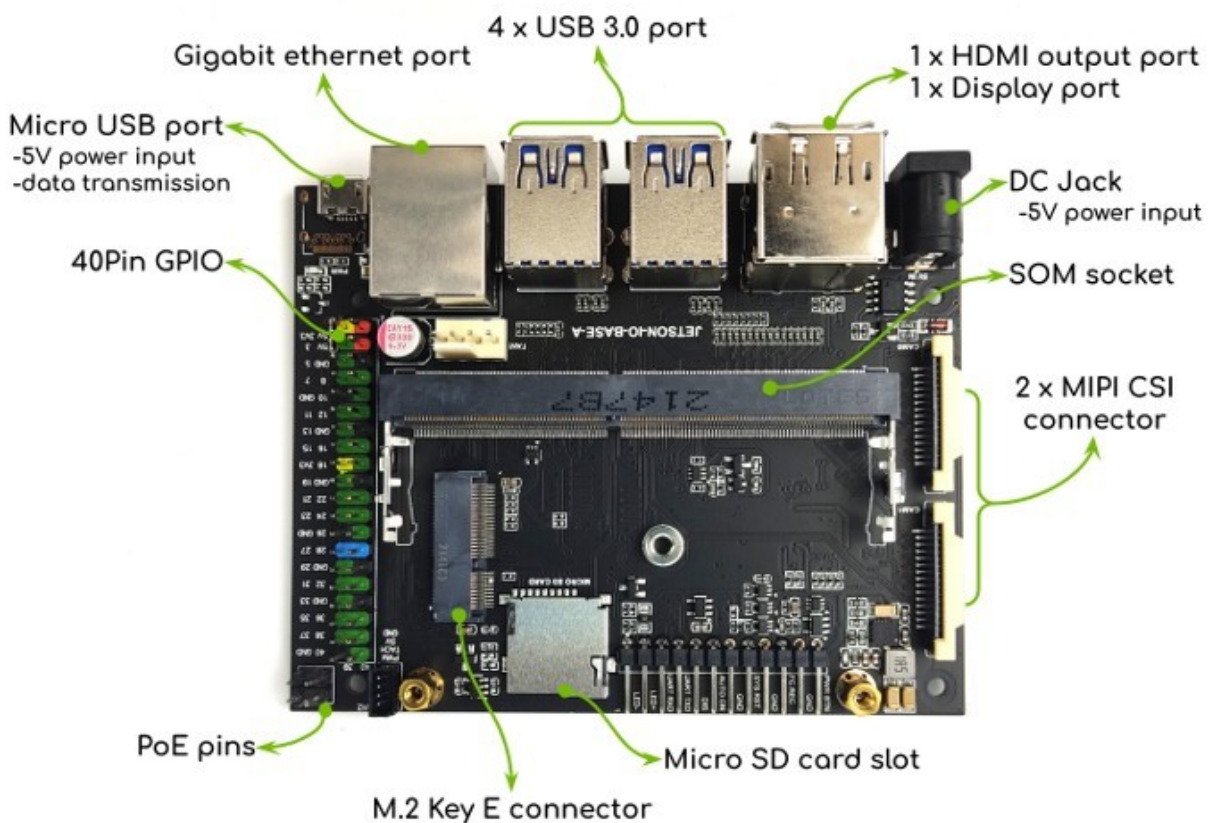


Figure 2.6. NVIDIA Jetson Nano Developer Kit B01 used as the edge-inference platform.

Edge processing can reduce dependence on a network connection and can reduce transmission of raw biometric images when the surrounding application is designed accordingly [20]. It does not by itself guarantee privacy or security: template protection, access control, encryption, logging, update policy, and presentation-attack resistance remain separate requirements.

Jetson Nano deployment is constrained by memory capacity, compute throughput, and software compatibility [25], [38]. Consequently, the embedding network is evaluated separately from localization and normalization, and network-only latency is distinguished from end-to-end latency. The measured FP32/FP16 comparison is reported in Chapter 5 without imposing an unverified real-time requirement.

2.15 Evaluation Metrics

Evaluating an iris verification system requires metrics suitable for the open-set matching task [22]. In contrast to closed-set classification where accuracy, precision, and F1-score are standard, biometric verification relies on quantifying matching errors. The primary metrics include False Acceptance Rate (FAR), False Rejection Rate (FRR), True Acceptance Rate (TAR), Equal Error Rate (EER), and the Area Under the Receiver Operating Characteristic Curve (AUC) [22].

1. Cosine similarity: The similarity score between two 512-dimensional feature embeddings u and v is

$$s(u, v) = \frac{u^T v}{\|u\|_2 \|v\|_2}$$

(2.7)

where the numerator is the dot product and the denominator contains the L2 norms. Because the model outputs L2-normalized embeddings, the implemented score reduces to the dot product. The score lies in $[-1, 1]$, with larger values indicating stronger angular agreement.

2. Genuine and impostor pairs: Verification performance is evaluated by comparing pairs of iris embeddings. A genuine pair consists of two samples from the same subject, whereas an impostor pair consists of samples from different subjects. The distribution of genuine similarity scores is compared against the distribution of impostor similarity scores to evaluate the separation margin between classes.
3. False Acceptance Rate (FAR): An impostor pair is falsely accepted when its similarity score is at least the decision threshold τ . For the finite set of impostor pairs,

$$FAR(\tau) = \frac{\# \{s_k^{imp} \geq \tau\}}{N_{imp}}$$

(2.8)

A lower FAR reduces false matches in security-sensitive applications.

4. False Rejection Rate (FRR): A genuine pair is falsely rejected when its similarity score is below τ . For the finite set of genuine pairs,

$$FRR(\tau) = \frac{\# \{s_k^{gen} < \tau\}}{N_{gen}}$$

(2.9)

A high FRR reduces usability by rejecting more legitimate comparisons.

5. True Acceptance Rate (TAR): A genuine pair is correctly accepted when its similarity score is at least τ . Therefore,

$$TAR(\tau) = \frac{\# \{s_k^{gen} \geq \tau\}}{N_{gen}} = 1 - FRR(\tau)$$

(2.10)

Verification performance is commonly reported as TAR at a controlled FAR, such as TAR at FAR = 0.1%.

6. Receiver Operating Characteristic (ROC) and Area Under the Curve (AUC): The ROC curve plots the TAR against the FAR across all possible decision thresholds τ . The AUC is the integral of the ROC curve, providing a single scalar value from 0 to 1 that indicates overall discrimination capability. An AUC of 1.0 represents perfect separation between genuine and impostor distributions, while 0.5 denotes a random classifier.
7. Equal Error Rate (EER): On the finite set T of evaluated thresholds, the implementation selects the threshold at which the empirical FAR and FRR are closest:

$$\tau_{EER} = \arg \min_{\tau \in T} |FAR(\tau) - FRR(\tau)|$$

(2.11)

The reported EER is the mean of the two rates at that threshold:

$$EER = \frac{FAR(\tau_{EER}) + FRR(\tau_{EER})}{2}$$

(2.12)

A lower EER indicates better separation under this summary criterion, but the EER threshold is not automatically appropriate for deployment [22], [35].

8. Validation-selected threshold: The operational threshold must be selected on validation scores before testing. Let T denote the finite candidate thresholds computed from validation scores. The thresholds satisfying the target FAR constraint form the feasible set

$$T_{val} = \{\tau \in T : FAR_{val}(\tau) \leq FAR_{target}\}$$

(2.13)

Within that feasible set, the implementation selects the threshold with maximum validation TAR:

$$\tau_{val} = \arg \max_{\tau \in T_{val}} TAR_{val}(\tau)$$

(2.14)

The target FAR is commonly set to 0.1% or 0.01%. After selection, the threshold is frozen and applied unchanged to the test set.

9. Network-only versus end-to-end latency and throughput: Network-only latency times the embedding engine, whereas end-to-end latency also includes acquisition, localization, normalization, data transfer, and matching. Throughput is reported in frames per second under a stated batch and stream configuration. These quantities must be reported separately because optimizing the network does not remove preprocessing cost.
10. Memory footprint and model size: The model size is determined by the number of trainable parameters and the storage required for the weights (e.g., in megabytes). Memory footprint refers to the runtime random-access memory (RAM) and graphics memory (VRAM) consumed during inference. Minimizing these metrics is crucial for deploying deep learning models on edge devices like the Jetson Nano, which have limited memory resources.

Verification metrics reflect the security and usability profile of the system [22]. EER summarizes the point where the two empirical error rates are closest, while TAR at a controlled FAR (for example, TAR at FAR = 0.1%) reports recognition performance at a stated security operating point. This project uses EER, AUC, and verification rates at thresholds selected from the validation partition to avoid test-set threshold tuning.

2.16 Related Work Synthesis, Research Gap, and Chapter Summary

Classical IrisCode and log-Gabor systems establish interpretable normalized-strip baselines [3], [4], [6], [7], [33], whereas deep methods learn continuous representations from polar or less constrained ocular inputs [8]–[11], [15], [29], [30]. Occlusion estimation and multi-scale fusion offer plausible reliability mechanisms [16], [23], [34], but published results across datasets and sensors are not directly comparable without aligned acquisition and subject-disjoint protocols [21], [22], [35]–[37]. The research gap addressed here is therefore deliberately narrow: a controlled comparison of no masking, deterministic soft angular masking, and MSFF under one frozen protocol; an evaluation-only classical reference; a traceable embedding pipeline; and measured PyTorch FP32/TensorRT FP32/TensorRT FP16 deployment on one Jetson Nano. These single-run results characterize this implementation and do not establish universal or statistically significant superiority.

CHAPTER 3. SYSTEM ARCHITECTURE AND TECHNOLOGY STACK

3.1 System Architecture Overview

The iris recognition system follows a sequential multi-stage pipeline. Each input eye image is converted to grayscale before the pupil and outer iris boundaries are localized. The detected iris region is then transformed into a fixed-size polar image using Daugman’s rubber sheet model. A deep neural network extracts an embedding from the normalized image, and cosine similarity is used to compare embedding pairs. Verification performance is evaluated using EER, AUC, and TAR at selected FAR thresholds.

For edge deployment, the feature-extraction network is exported to ONNX [26] and compiled into TensorRT FP32 and FP16 engines on the NVIDIA Jetson Nano [24], [25], [38], [39]. PyTorch FP32 is retained as the reference implementation. Image preprocessing, including grayscale conversion, boundary localization, and Daugman normalization, remains outside the exported network; therefore, preprocessing latency and neural-network inference latency are measured separately.

System Architecture Overview

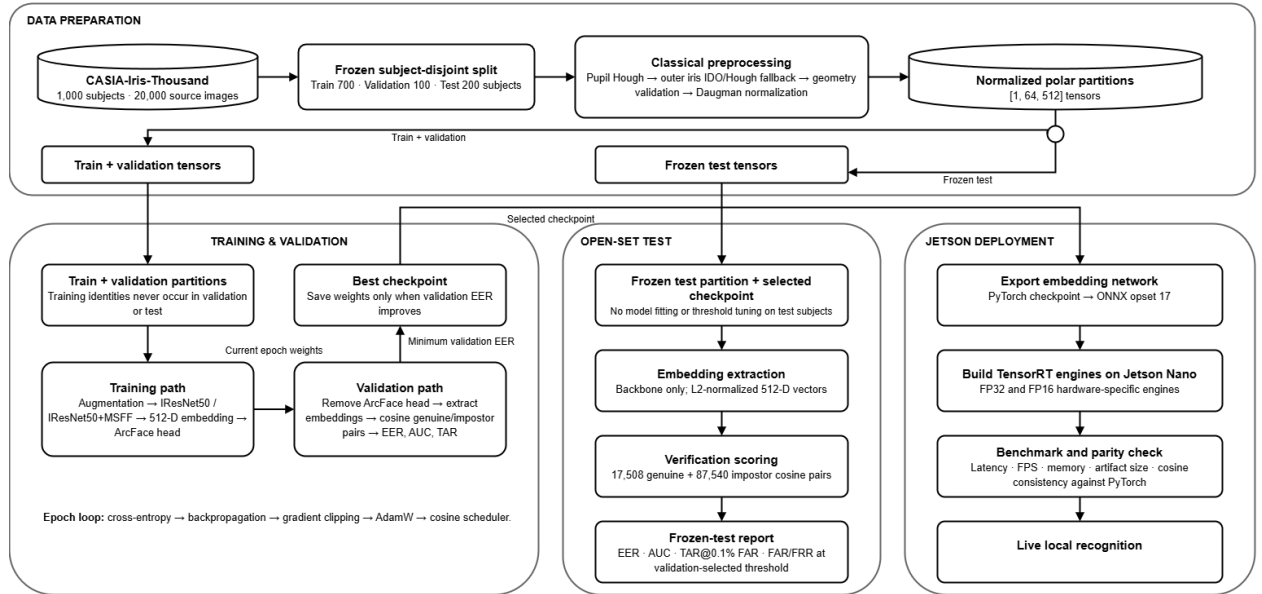


Figure 3.1. Schematic overview of the iris recognition system pipeline.

3.2 Software Architecture

The software architecture of the system is modularized into distinct functional components with clear boundaries to ensure maintainability. Table 3.1 outlines the primary software modules and their respective responsibilities.

Table 3.1. Primary software modules and functional responsibilities.

Module	Representative Files	Functional Responsibility
Data Management	src/data.py	Explores the dataset, generates or loads split configurations, and ingests normalized polar images.

Preprocessing	src/preprocessing.py	Performs pupil and iris boundary localization alongside Daugman rubber sheet normalization.
Model Architecture	src/models.py	Defines neural backbone configurations including IResNet50, MobileFaceNet, MobileNetV2, and Multi-Scale Feature Fusion (MSFF) modules.
Training Pipeline	experiments/train.py	Manages ArcFace-guided optimization, hyperparameter configuration, and checkpoint selection.
Evaluation	experiments/evaluate.py; src/metrics.py	Extracts feature embeddings and calculates performance metrics including EER, AUC, TAR, FAR, and FRR.
Deployment	experiments/export_onnx.py	Exports the PyTorch models to ONNX format and structures the pipeline for inference optimization.

3.3 Technology Stack

Table 3.2 enumerates the primary technologies integrated from the codebase, detailing their specific roles and validation statuses within the project framework.

Table 3.2. System technology stack and verification status.

Component	Role	Status
Python	Implementation language	Utilized
PyTorch	Model training, evaluation, and ONNX graph export	Completed
TorchVision	Baseline MobileNetV2 architecture implementation	Completed
OpenCV	Image ingestion, Hough Circle Transform localization, and polar mapping	Completed
scikit-learn	Computation of ROC, AUC, and verification metrics	Completed
ONNX	Intermediate model exchange representation	Completed export pipeline
Component	Role	Status
TensorRT	Inference engine optimization	Verified (FP32 and FP16 engines compiled on Jetson Nano)
Jetson Nano	Target edge execution platform	Verified (FP32 and FP16 engines successfully deployed and evaluated)

3.4 System Data Flow

The system data flow initiates with the extraction of iris image paths from structured CSV metadata files containing subject-specific partitions. The custom IrisDataset class ingests

the grayscale images and executes a preprocessing sequence that performs pupil/iris localization and geometric normalization. Image samples failing to satisfy the defined geometric constraints (e.g., corrupted inputs or localization failures) are systematically excluded to ensure that only verified, processed samples are loaded into memory.

During the training phase, the DataLoader batches the data into input tensors of dimensions [B, 1, 64, 512]. The feature extraction network maps the normalized polar representations to 512-dimensional normalized embedding vectors. Subsequently, the ArcFace classification layer computes angular logit scores relative to the subject-eye identity labels, and the resulting angular cross-entropy loss is backpropagated to optimize the network weights.

Conversely, during the validation and verification phase, the system operates exclusively using the feature extraction model, bypassing the ArcFace classifier. Cosine similarity is computed between embedding pairs to generate genuine and impostor score distributions. These distributions are subsequently analyzed to plot Receiver Operating Characteristic (ROC) curves and to derive verification metrics, including EER, AUC, TAR, FAR, and FRR.

3.5 Training Architecture

The training pipeline comprises the following sequential procedures:

1. Establish a frozen dataset partition to support reproducible training and validation subsets.
2. Preload the partitioned iris dataset into memory using the custom dataset class.
3. Initialize the data loader to manage batch generation and worker scheduling.
4. Apply data augmentation techniques to the batched inputs to enhance model generalization.
5. Forward-pass inputs through the neural backbone to extract feature embeddings.
6. Compute angular margins and logit representations using the ArcFace head.
7. Calculate training loss using the angular cross-entropy loss function.
8. Execute backpropagation and update network parameters using the AdamW optimizer.
9. Adjust learning rates dynamically using a cosine annealing schedule.
10. Extract embeddings from the validation set under an open-set protocol.
11. Compute open-set verification performance metrics on the validation dataset.
12. Identify and save the optimal model checkpoint based on the lowest validation EER.

The optimal model checkpoint is selected based on the lowest open-set Equal Error Rate (val_eer) achieved on the independent validation dataset, rather than the training classification accuracy (train_acc). This selection criteria aligns with the requirements of open-set verification, where the system must evaluate novel identities not present during the training phase.

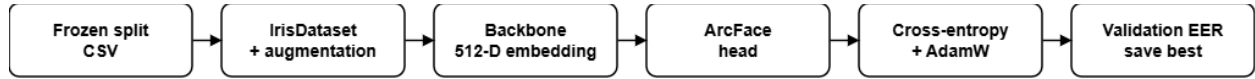


Figure 3.2. Detailed schematic of the training and checkpoint selection architecture.

3.6 Inference and Jetson Nano Deployment Architecture

The ONNX graph contains the deep feature-extraction network only. Camera acquisition, grayscale conversion, pupil and iris localization, Daugman normalization, angular masking, input normalization, and cosine matching remain in the Jetson control application. This boundary is important because an ONNX export of the network does not by itself implement an iris-recognition product; it implements only the learned mapping from a normalized iris tensor to an embedding vector.

The audited control layer is implemented as a Python Flask service with a browser client. The backend wrapper loads the TensorRT engine and metadata, serializes inference for Jetson/PyCUDA stability, checks that the embedding is finite and non-zero, and returns preprocessing metadata. The local database stores normalized left- and right-eye templates and the recognition endpoint reports the score, threshold decision, optional same-side restriction, competing score, and margin result. These returned fields allow an operator to distinguish a valid high-score match from an ambiguous or rejected query.

At inference time, a camera frame first passes the acquisition and quality stages. The accepted eye crop is converted to the same normalized 64×512 polar representation used by the trained model. After input scaling, the backend executes one batch-one forward pass and returns an embedding e . The application computes $z = e / \|e\|$ and compares z with each permitted stored template. For normalized query and template vectors, the score is $s = zqzt$. The identity is selected by the largest valid score, but the decision is accepted only when $s \geq \tau$ and, when configured, the score margin over the second-best candidate is at least m . This is the mechanism by which the system decides that a result is sufficiently consistent with an enrolled identity; the neural network supplies a representation, while the application supplies the verification rule. The complete runtime sequence from camera acquisition to the threshold-based identity decision is summarized in Figure 3.3

Jetson Nano Live Recognition Flow

The network outputs an embedding, acceptance requires both similarity threshold and optional best-versus-second-best margin.

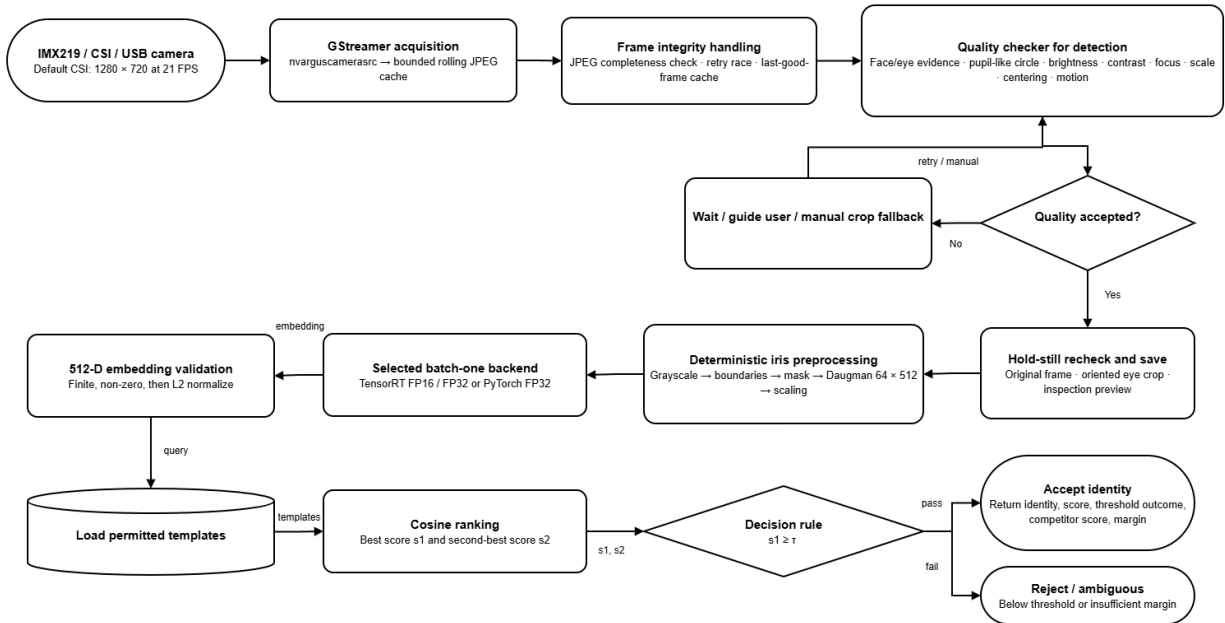


Figure 3.3. Jetson Nano live recognition flow from camera acquisition to the browser-reported verification decision.

For the IMX219/CSI path, “nv_argus_camera_src” feeds a persistent GStreamer preview. The default 1280 × 720, 21-fps configuration can be changed through camera settings without changing the embedding model. A bounded rolling JPEG cache, completeness checks, retry logic, and a last-good-frame cache make the browser stream tolerant of the file replacement race created by multifilesink. The browser shows the unmodified NoIR view beside grayscale or NIR-contrast views so that the operator can distinguish sensor appearance from recognition preprocessing.

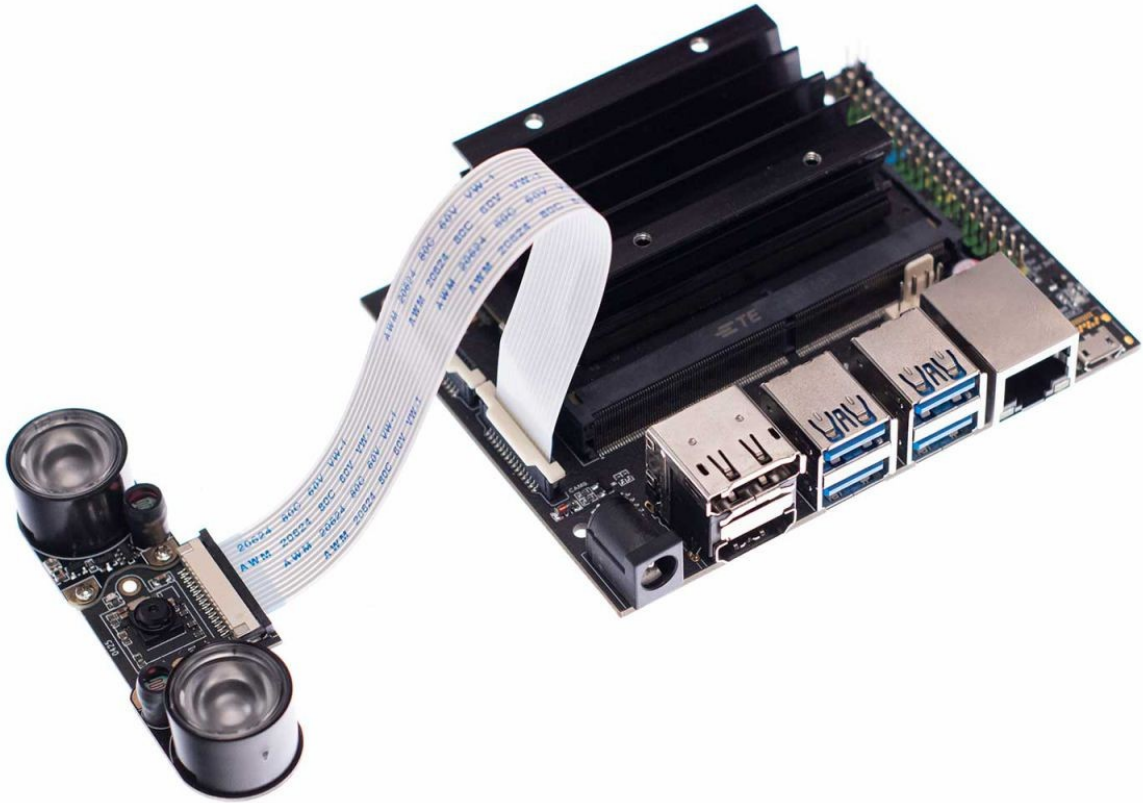


Figure 3.4. Hardware configuration for live iris-image acquisition and edge inference.

The browser provides automatic and manual capture. Automatic capture uses face and eye detection, pupil-like circular evidence, brightness, contrast, focus, eye size, centering, and motion stability to produce a heuristic quality score. A frame is saved only after the score and explicit quality conditions pass and the operator remains still for the hold interval. The manual photo-and-crop route is retained as a recovery path. These quality tests control which frames enter inference, but they are not a trained liveness detector and do not establish camera-specific biometric accuracy.

Enrollment accepts one quality-checked capture for each eye, creates normalized embeddings, and stores them with the identity and eye-side labels. Recognition follows the same preprocessing and embedding path, ranks the permitted templates, and reports the identity, score, threshold outcome, competing score, and optional margin outcome. A high cosine value is therefore evidence of representation similarity, not a probability of identity and not proof of correctness without a labelled evaluation protocol. This distinction is retained throughout the thesis to avoid overstating what the live demonstration measures.

3.7 Functional Requirements

The iris recognition system is designed to satisfy the following functional requirements:

- Ingest and parse the iris dataset according to subject identities and specific eye labels (left or right).

- Generate or load independent subject-wise partitions for training, validation, and testing phases.
- Perform pupil and iris boundary localization, and normalize the iris region to a 64×512 polar representation.
- Train the feature extraction backbone guided by ArcFace loss to produce discriminative embedding vectors.
- Evaluate open-set verification performance and log reproducible results to disk.
- Export the trained feature extraction neural network to the ONNX format.

3.8 Non-functional Requirements

Additionally, the system must adhere to the following key non-functional requirements:

- **Reproducibility:** Ensure consistency across runs by fixing random seeds, dataset partitions, and training configurations.
- **Integrity:** Report only verified empirical metrics and explicitly delineate completed deployment phases.
- **Data Privacy:** Protect sensitive assets by qualifying privacy claims and avoiding public storage of raw iris images.
- **Deployability:** Structure the pipeline with an ONNX export pathway to facilitate local edge inference.
- **Maintainability:** Modularize the codebase into clear components covering data, models, training, evaluation, and deployment.

3.9 Chapter Summary

Chapter 3 has detailed the system's software modularization and data flow. The training and evaluation pipelines have been established, and the model has been exported to the ONNX format. Furthermore, deployment on the target NVIDIA Jetson Nano hardware has been verified, demonstrating successful compilation and execution of PyTorch FP32, TensorRT FP32, and TensorRT FP16 engines. Based on this system architecture, the subsequent chapter will present the mathematical foundations and specific technical implementation details.

CHAPTER 4. METHODOLOGY AND IMPLEMENTATION

4.1 Implementation Methodology Overview

The implemented method combines three primary technical stages: classical digital image processing for iris segmentation and normalization; deep learning for extracting highly discriminative feature vectors; and verification evaluation to quantitatively measure recognition performance. Rather than directly resizing the original ocular image, which introduces geometric distortion, the system localizes the pupil and outer iris boundaries and then applies Daugman's rubber sheet model to normalize the annular region into a rectangular polar representation. This approach ensures that the deep neural network receives input data concentrated solely on the biologically salient iris texture.

The deep neural network is trained using the ArcFace loss function based on the subject-eye classes in the training set. During the evaluation phase, the system operates under an open-set protocol: the subjects in the test set are completely independent of and do not appear in the training set. The feature vectors extracted from the test set are paired into genuine matches and impostor matches to compute quantitative verification metrics.

4.2 Dataset

Experiments use the CASIA-Iris-Thousand subset distributed within CASIA-IrisV4 [21]. The frozen metadata contains 20,000 source images from 1,000 subjects, with left and right eyes treated as 2,000 subject-eye classes. The dataset is used under its institutional access conditions; raw iris images are not redistributed with the public reproducibility materials.

The class label for training is defined as a subject-eye class (under the assumption that the iris patterns of the left and right eyes of the same individual are statistically independent). The dataset splitting is executed strictly under a subject-disjoint protocol to prevent images of the same subject from appearing in both the training and testing partitions. The distribution origin and licensing of the dataset replica must be verified by the author prior to report publication.

4.3 Data Preprocessing

The ocular image preprocessing pipeline is established with the following sequential steps:

1. Read the image using OpenCV in grayscale mode.
2. Detect the pupil boundary.
3. Detects the outer iris boundary.
4. Perform geometric validation between the pupil and iris circles.
5. Normalize the iris region to a polar representation.
6. Filter out failed preprocessing samples.

7. Convert the normalized polar image to a tensor.
8. Scale pixel intensity values to the range [0, 1].
9. Standardize the tensor using norm_mean = 0.449 and norm_std = 0.226.
10. Apply a soft angular mask if enabled in the configuration.
11. During training, apply data augmentations including horizontal angular roll (aug_roll_frac = 0.0625), brightness and contrast adjustment (probability = 0.7), random erasing (probability = 0.15), and a randomized angular mask bank when soft angular masking is active.

4.4 Pupil and Iris Localization

The pupil is localized using the circle Hough transform via the cv2.HoughCircles function after the grayscale image is smoothed with a median filter. The detailed parameter configurations are presented in Table 4.1:

Table 4.1. Pupil detection parameters using the circle Hough transform.

Parameter	Value
Median filter kernel size	7
Detection method	cv2.HOUGH_GRADIENT
dp	1
minDist	100
param1	100
param2	18
minRadius	15
maxRadius	80

The outer iris boundary is initially localized using the Integro-Differential Operator (IDO) algorithm. In the event that the primary detection is invalid or violates geometric constraints, the system automatically activates a fallback circle Hough transform (Hough fallback) with the parameters specified in Table 4.2:

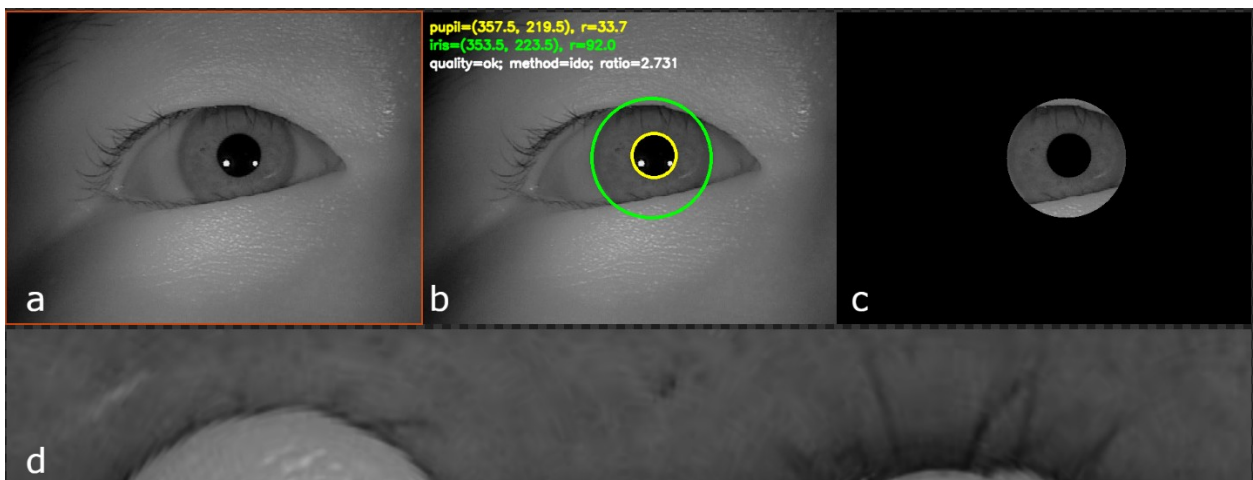


Figure 4.1. Representative preprocessing stages of the iris-recognition pipeline: (a) input grayscale eye image; (b) detected pupil boundary shown in yellow and outer iris boundary shown in green, together with localization metadata; (c) segmented iris annulus isolated from the surrounding eye region; and (d) normalized polar iris representation generated using Daugman’s rubber-sheet model.

Table 4.2. Outer iris boundary detection parameters using Hough fallback.

Fallback Parameter	Value
Median filter kernel size	7
dp	1.2
minDist	200
param1	100
param2	30
minRadius	$\text{int}(\text{pupil_radius} \times 1.5)$
maxRadius	$\text{int}(\text{pupil_radius} \times 4.0)$

A geometric constraint validation step is performed to filter out erroneous localization results. The validation criteria include: the ratio of the iris radius to the pupil radius must lie within the interval $[1.8, 5.0]$; the eccentric displacement between the pupil center and the iris center must not exceed $0.35 \times r_{\text{iris}}$ (where r_{iris} denotes the outer iris radius); and the boundary of the detected iris circle must not extend beyond the image borders by more than a specified margin.

During a preliminary check conducted in the experimental notebook, the system processed a random sample of 200 images, achieving a localization success rate of 100% (200/200 images) with a mean iris-to-pupil radius ratio of 2.86. It is critical to note that this represents a quick verification on a small subset and does not guarantee the preprocessing success rate across the entire official dataset.

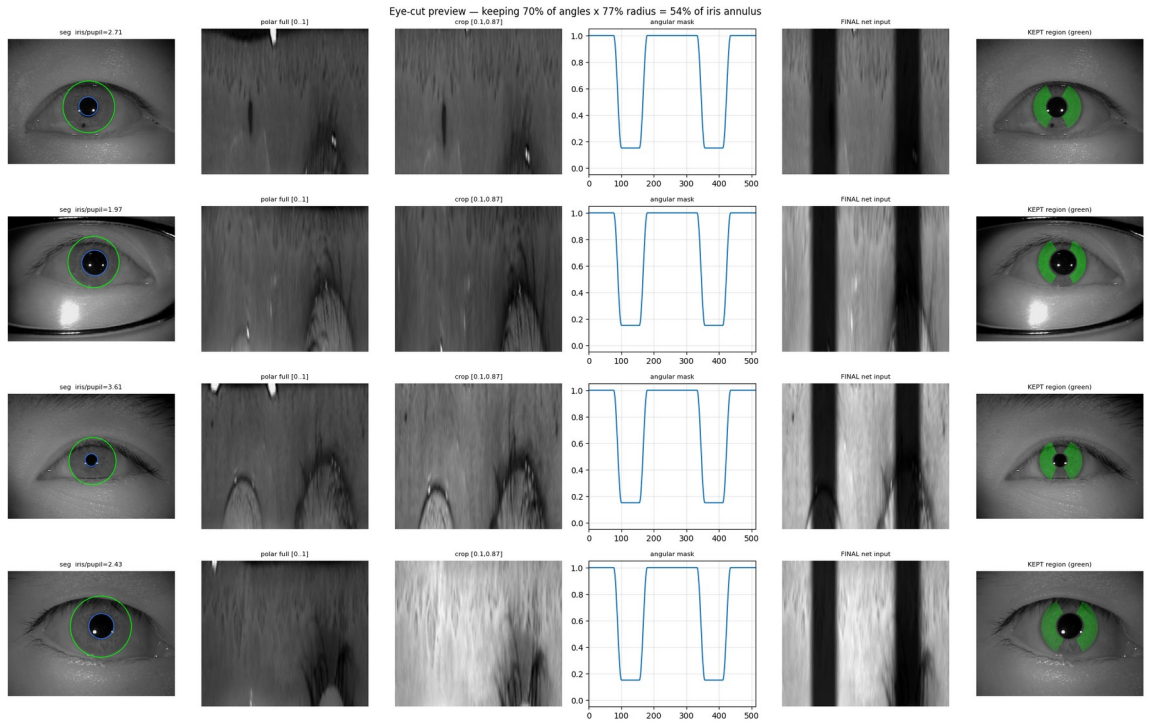


Figure 4.2. Preprocessing and localization preview from the experimental notebook.

Following the precise localization of the pupil and outer iris boundaries, the system applies a mapping based on Daugman's rubber sheet model to transform the annular iris region into a normalized rectangular polar representation. The configuration parameters for normalization are detailed in Table 4.3:

4.5 Daugman Normalization and Subject-Disjoint Splitting

Table 4.3. Iris normalization parameters using Daugman's model.

Parameter	Value
polar_height	64
polar_width	512
radial_inner	0.10
radial_outer	0.87
Interpolation method	cv2.INTER_LINEAR
Border mode	cv2.BORDER_REPLICATE

Setting the inner radial boundary to `radial_inner = 0.10` excludes the innermost iris region near the pupil, which is frequently corrupted by pupillary dilation noise or specular reflections. Similarly, limiting the outer radial boundary to `radial_outer = 0.87` mitigates localization errors near the sclera boundary, where occlusion by eyelids and eyelashes is common.

The dataset splitting is conducted strictly at the source-image metadata level prior to runtime localization and normalization, using a subject-disjoint protocol. Under this protocol, the identities of subjects in the training, validation, and test splits are completely separated to prevent identity leakage. The source split CSV files contain metadata mapping 14,000, 2,000, and 4,000 images representing 700, 100, and 200 subjects, respectively. The

existing project configuration CSV files confirm the complete absence of subject overlap among these partitions.

During runtime localization, the notebook aggregate preprocessing pipeline retained 19,642 of the 20,000 images, experiencing 358 localization failures. For the official classical validation and test set evaluations, preprocessing retained 1,959 of 2,000 validation images and 3,942 of 4,000 test images. Per-split retained counts for the deep learning models are not implied where they are unavailable, maintaining experimental transparency.

Table 4.4. Dataset partition statistics.

Split	Subjects	Subject-eye classes	Images	Images per class	Left / Right
Training	700	1,400	14,000	10	7,000 / 7,000
Validation	100	200	2,000	10	1,000 / 1,000
Test	200	400	4,000	10	2,000 / 2,000
Total	1,000	2,000	20,000	10	10,000 / 10,000

The configuration parameters for the dataset partitioning procedure are summarized in Table 4.5:

Table 4.5. Subject-disjoint dataset splitting configuration.

Parameter	Value
Seed	42
Training subject ratio	0.70
Validation subject ratio	0.10
Test subject ratio	0.20
min_samples	3

Subject-disjoint partitioning prevents the same identity from appearing in both training and evaluation. This supports evaluation on unseen subjects and avoids an obvious source of optimistic bias, although it does not by itself establish cross-sensor generalization [22].

4.6 Model Design

The implemented model architecture accepts a polar iris image tensor of size [B, 1, 64, 512] and outputs a 512-dimensional L2-normalized feature embedding vector.

Table 4.6. Architectural configurations and parameter counts of the evaluated models.

Model	Configuration	Total parameters	Trainable parameters	Status
IResNet50	Without MSFF	39,133,248	39,132,736	Completed (b1 and b3)
IResNet50 + MSFF	Fused layer3/layer4	43,918,144	43,917,632	Completed (ours configuration)
MobileFaceNet	Lightweight feature extractor	1,239,808	1,239,296	Implemented; baseline 4 incomplete
MobileNetV2	TorchVision variant	2,880,192	2,879,680	Code available; currently unused in

				baseline 4
--	--	--	--	------------

The ArcFace classifier head receives the 512-dimensional feature vector and maps it to output nodes corresponding to the number of training subject-eye classes. During the evaluation and inference phases, this classification head is removed; only the backbone embedding network is activated to perform cosine similarity matching between image pairs.

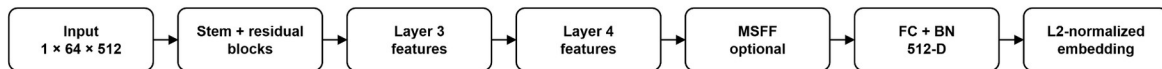


Figure 4.2. IResNet50 architecture with MSFF and the extracted feature vector.

Within the scope of this project, the term ArcIris designates the internal baseline configuration. This configuration represents an iris recognition model operating on Daugman-normalized polar images, utilizing a convolutional network backbone to extract 512-dimensional feature embeddings, optimized using the ArcFace loss function. The general processing procedure is established as follows:

4.7 Training Configuration

Table 4.7. Hyperparameter configuration for model training.

Category	Parameter	Value
Data	polar_height	64
Data	polar_width	512
Data	norm_mean	0.449
Data	norm_std	0.226
Training	batch_size	128
Training	epochs	40 (target in YAML; 36 executed in prototype notebook)
Training	Optimizer	AdamW
Training	Learning rate	0.001
Training	Weight decay	0.0003
Training	Scheduler	CosineAnnealingLR
Training	eta_min	1e-6
Training	Label smoothing	0.02
Training	Gradient clipping	5.0
Training	AMP	True if CUDA is available
ArcFace	Margin	0.25
ArcFace	Scale	64.0
Regularization	Dropout	0.35
Early stopping	Patience	8
Early stopping	Min delta	0.02
Evaluation	Validation pairs	100,000

Evaluation	Test pairs	200,000
Evaluation	Target FAR	0.001
Evaluation	Impostor multiplier	5

4.8 Training Procedure

The detailed model training procedure implemented in `experiments/train.py` consists of the following key steps:

1. Load the YAML configuration file.
2. Set the global random seed for reproducibility.
3. Identify the target hardware device (CPU or GPU).
4. Load or generate the subject-disjoint split CSV metadata.
5. Preload and preprocess the training and validation datasets.
6. Initialize the backbone network and the ArcFace classifier head.
7. Instantiate `CrossEntropyLoss`, the `AdamW` optimizer, and the `CosineAnnealingLR` scheduler.
8. For each epoch, execute the training loop: forward pass, compute ArcFace logits, calculate loss, perform backpropagation, clip gradients, perform optimizer and scheduler updates, extract validation embeddings, compute EER, AUC, and TAR@FAR, and save checkpoints when validation EER improves.
9. Log metrics to `training_log.csv`.
10. Save the resume state if executed with the `--resume-state` flag.

4.9 Verification Evaluation Procedure

The system verification performance evaluation procedure established in `experiments/evaluate.py` comprises the following steps:

1. Load the configuration YAML and the `best_model.pth` checkpoint.
2. Re-initialize the backbone network architecture.
3. Load the frozen validation and test partition metadata.
4. Preload and preprocess the validation and test datasets.
5. Extract normalized embedding vectors for each image.
6. Sample genuine and impostor pairs according to the protocol. The baseline models (b1 and b3) and the implemented model (ours) share a frozen subject-exclusive split and pair protocol, yielding exactly 17,508 genuine pairs and 87,540 impostor pairs for the test set of each configuration.
7. Compute cosine similarity scores for all evaluation pairs.

8. Calculate EER, AUC, TAR at 0.1% FAR, the decision threshold, and corresponding FAR/FRR metrics.
9. Save metrics JSON, embeddings NPZ, score distribution CSV, ROC curve plot, and score histogram.

This procedure implements open-set verification because test subjects are disjoint from training subjects. The validation partition is used for checkpoint and operating-threshold selection, while final test curves and validation-threshold error rates are reported separately.

4.10 Grad-CAM++ Visualization

In the experimental notebook, the Grad-CAM++ algorithm is implemented via a wrapper to capture cosine similarity logits from the ArcFace head. For the IResNet50 configuration, the target layer is selected as the final convolutional layer of the layer3 block (layer3[-1]) to preserve better spatial resolution compared to deeper layers. The resulting heatmap on the polar image is overlaid onto the normalized polar strip and then back-projected to the original Cartesian coordinate system using the geometric parameters from Daugman's model.

Although the repository does not currently package an independent Grad-CAM++ evaluation script within the experiments/ directory, qualitative visualizations from the original notebook have been exported. Consequently, this visualization component is presented as notebook-derived qualitative evidence to interpret model behavior, rather than serving as an independent quantitative benchmark.

4.11 Model Export

The experiments/export_onnx.py script exports the feature extraction backbone network to the ONNX format. This script utilizes a wrapper to ensure that the model output is restricted to the 512-dimensional feature embedding, discarding intermediate feature maps. The input and output configurations of the exported ONNX file are summarized in Table 4.8:

Table 4.8. ONNX input and output specifications.

Component	Value
Input name	iris_polar
Input shape	[batch_size, 1, 64, 512]
Output name	embedding
Output shape	[batch_size, 512]
Dynamic axes	batch_size
Opset versions tested	17, 13, 12

The final embedding network was exported to ONNX with opset 17, producing a 167.6 MB graph that passed ONNX structural validation. The preserved project artifacts do not contain a separate full ONNX Runtime parity report; numerical parity evidence in this thesis is therefore limited to the measured PyTorch-versus-TensorRT outputs on the Jetson deployment inputs.

4.12 TensorRT Optimization

The inference optimization pipeline consists of the following sequential stages:

1. Export the PyTorch model checkpoint to ONNX format.
2. Verify the ONNX graph structure.
3. Where an ONNX Runtime environment is available, compare its outputs with PyTorch on fixed inputs; record the absence of this check when the result is not preserved.
4. Compile the ONNX model to a TensorRT engine on the Jetson Nano.
5. Measure execution latency after a warm-up period.
6. Measure throughput (FPS), memory footprint, model size, and verify power mode.
7. Verify embedding divergence and its impact on metrics (EER, FAR, FRR).



Figure 4.3. Export and optimization workflow: PyTorch to ONNX to TensorRT on Jetson.

4.13 Jetson Nano Deployment Procedure

A complete physical deployment integrates camera acquisition, grayscale conversion, pupil and iris localization, Daugman normalization, input scaling, TensorRT inference, cosine matching, and threshold-based decision making. The ONNX graph contains the embedding network only; the remaining stages are executed by the Jetson control application. This division keeps the exported graph small and makes it possible to compare PyTorch, TensorRT FP32, and TensorRT FP16 while holding the camera and preprocessing path constant.

The application is a Python 3.6-compatible Flask service with a browser client. At startup it loads the selected engine and its metadata, checks configured model and database paths, initializes the camera pipeline, and exposes health, camera, quality, capture, registration, and recognition endpoints. The default CSI configuration uses the IMX219-style Jetson camera path at 1280×720 and 21 frames per second. USB and alternate CSI sources can be selected without changing the embedding backend, so the acquisition layer is replaceable while the matching logic remains stable.

For continuous preview, GStreamer uses `nvarguscamerasrc` to acquire frames from the CSI sensor and writes a bounded rolling sequence of JPEG files. The server validates file size and JPEG completeness, retries the short replacement race produced by `multifilesink`, and serves the last valid frame when a new file is temporarily unavailable. This is an engineering requirement rather than a cosmetic detail: without the validation and last-good-frame policy, the browser can display a broken image while the camera is still operating. Flask request handling is kept single-threaded and TensorRT/PyCUDA access is serialized by a backend lock for device stability.

The automatic capture procedure is a quality gate rather than a blind snapshot. A background worker analyzes the newest frame using OpenCV face and eye detection, pupil-like circular evidence, brightness, contrast, focus, eye size, centering, and inter-frame stability. The resulting heuristic score is bounded between 0 and 100. The default score threshold is 72, and explicit rejection conditions cover missing eye evidence, implausible scale, excessive darkness or brightness, insufficient sharpness, weak pupil evidence, and motion. After a short hold-still interval, the server repeats the check and saves the original frame, the oriented eye crop, and a grayscale CLAHE inspection preview. This prevents an image that merely looks acceptable in one preview frame from entering the recognition path without a second validation.

The accepted crop is then processed by the same deterministic stages used by the benchmark: grayscale conversion, boundary estimation, masking of unreliable angular regions, Daugman rubber-sheet normalization to 64×512 , and input normalization. The resulting tensor is passed to the selected backend at batch size one. The network returns an embedding rather than an identity. The controller checks the embedding norm, normalizes it, and calculates cosine similarity against the stored templates. The system reports the best identity only when the score passes the configured threshold and, when enabled, the best-versus-second-best margin. Thus, the application “knows” that a result is acceptable because the complete decision rule is satisfied; it does not infer correctness from the camera preview alone.

The registration workflow requires a name and two accepted captures, one for each eye. The backend normalizes both embeddings and stores them in the versioned local JSON database with their eye-side labels. Recognition can compare both eye sides or restrict the search to the known query side. Same-side matching reduces cross-eye comparisons, while margin rejection reduces ambiguous decisions when two identities obtain similar scores. If automatic detection fails, the operator can take a manual photograph and draw a crop before sending it through the same backend; the fallback changes acquisition, not the neural decision rule.

The deployment is deliberately separated from the neural training protocol. The browser application loads a frozen checkpoint or engine and never retrains it. The live interface demonstrates acquisition, preprocessing, enrollment, and decision control, whereas the benchmark reports fixed CASIA identities, frozen pairs, and machine-generated metrics. Consequently, a successful browser match demonstrates that the end-to-end software path is operational, but it must not be described as an additional CASIA accuracy result or as validated IMX219 biometric performance.

The camera demonstration is evaluated separately from the three-mode performance benchmark. PyTorch FP32, TensorRT FP32, and TensorRT FP16 use the same batch-one input and shared preprocessing; their network latency, end-to-end latency, throughput, memory, artifact size, and cosine parity are reported in Section 5.10. This separation makes the Jetson optimization claim auditable: the backend changes, while the acquisition and matching protocol remain fixed.

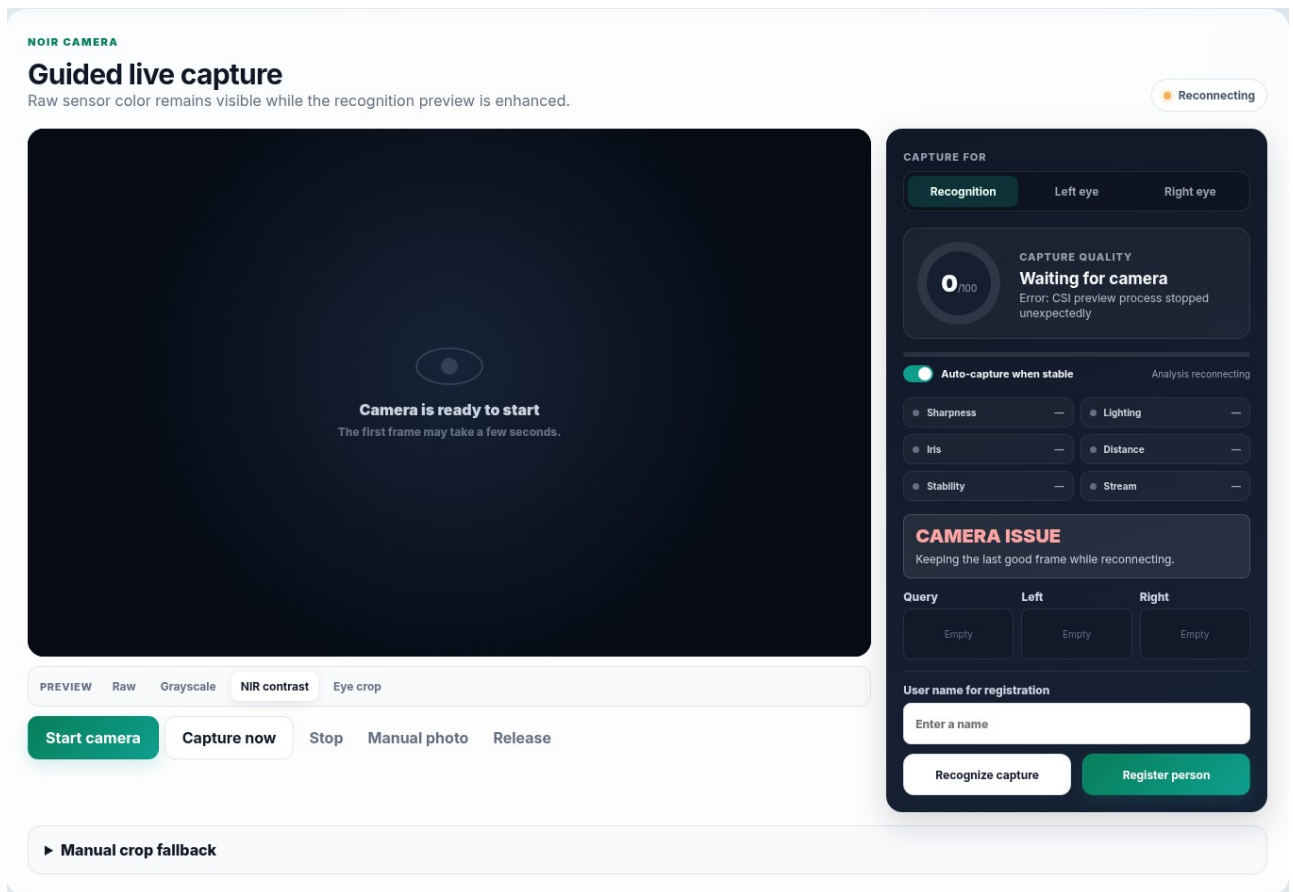


Figure 4.3. Fresh static render of the IMX219 NoIR live-capture interface, showing raw and processed previews, quality guidance, and the three-step capture flow.

4.14 Chapter Summary

Chapter 4 has detailed the methodology and implementation verified from the repository and experimental notebook. The system incorporates ocular image preprocessing, Daugman polar normalization, model training with ArcFace, open-set verification evaluation, Grad-CAM++ model interpretability, ONNX model export, TensorRT engine compilation, and physical benchmarking on the Jetson Nano B01. The following chapter will present, analyze, and discuss the quantitative experimental results obtained from these procedures.

CHAPTER 5. EXPERIMENTAL RESULTS AND EVALUATION

5.1 Experimental Environment

The experimental environment is thoroughly characterized based on configuration parameters and operational logs traceable from the project repository. Neural network training and benchmarking of the baseline configurations (b1 and b3) and the proposed model were conducted within the Kaggle platform environment. For prototype development and initial visualization, local Jupyter notebook sessions utilizing Python 3.12.12 were employed. Table 5.1 details the specific experimental environments, their verification statuses, and their roles in the evaluation workflow.

Table 5.1. Experimental environments and verification status.

Environment	Verifiable Information	Verification Status
Training & Benchmarking	Kaggle; configuration files and result outputs for baseline configurations (b1, b3)	Test results verified
Prototype Notebooks	Python 3.12.12 (as specified in notebook metadata)	Visual outputs and preliminary scores generated
Local Environment	Source code repository, local configurations, and summary reports	Not utilized for official baseline benchmarking
Jetson Nano B01	Target edge deployment hardware (MAXN mode)	Physical benchmark logs and precision comparisons verified

The neural models were trained with PyTorch and CUDA in GPU-enabled Kaggle environments. Edge measurements were collected on a Jetson Nano B01 in MAXN mode with the recorded PyTorch and TensorRT 8.2 software stack. These environments support the reported training and deployment experiments but do not by themselves establish portability to other GPUs or Jetson software versions.

5.2 Dataset Accounting

The evaluation framework employs a strict subject-exclusive split derived from the CASIA-Iris-Thousand dataset. The dataset comprises 1,000 independent subjects, corresponding to 2,000 distinct eye classes (left and right eyes for each subject), totaling 20,000 source images. To ensure strict subject-disjoint verification, the subject identities in the training, validation, and testing partitions are completely non-overlapping. Table 5.2 outlines the partitioning strategy, including the number of subjects, classes, images, and respective roles of each split.

Table 5.2. Dataset partitioning and statistics.

Partition	Subjects	Eye Classes	Images	Role
Training	700	1,400	14,000	ArcFace representation learning

Validation	100	200	2,000	Checkpoint selection and threshold tuning
Testing	200	400	4,000	Final model evaluation
Total	1,000	2,000	20,000	Complete subject-disjoint split

5.3 Iris Localization and Normalization Results

The preprocessing pipeline successfully localized and normalized 19,642 iris images out of the 20,000 source images, representing a retention rate of 98.21%. The remaining 358 images were excluded due to localization failures, primarily caused by pupil boundary detection errors, extreme eyelid occlusions, or invalid iris geometry. This filtering limitation represents an operational constraint of the traditional segmentation module. For the non-neural classical baseline, 1,959 validation and 3,942 test images were successfully processed and retained. Table 5.3 summarizes the preprocessing results, while the visual extraction and polar normalization procedures are illustrated in Fig. 4.1.

Table 5.3. Preprocessing statistics and retention rate.

Metri	Value	Source
Total Source Images	20,000	Fixed dataset split
Successfully Preprocessed Images	19,642	Output of iris_baseline.ipynb
Failed Preprocessing Images	358	Output of iris_baseline.ipynb
Retention Rate	98.21%	Calculated as 19,642 / 20,000

It is important to emphasize that these preprocessing statistics represent aggregate notebook measurements and serve to evaluate the general localization framework rather than specific model training instances. To establish rigorous comparisons, future work must log run-specific preprocessing rates under independent seed configurations. Operational limitations of classical Daugman-style segmentation are further discussed under model comparison.

5.4 Training Results

The final artifact set includes training logs for b1, b3, and the MSFF configuration, with epoch, training loss, validation EER/AUC/TAR, learning rate, and duration fields. Because Figure 5.1 in the earlier draft originated from a historical notebook checkpoint, it has been replaced by curves generated from the three collected final logs; test metrics are not used for checkpoint selection.

Figure 5.1 compares the validation EER trajectories recorded for the three final neural runs. The curves describe optimization behavior under one run per configuration and should not be interpreted as confidence bands. Differences in the epoch of the best checkpoint reflect the validation-based selection rule rather than a test-set choice.

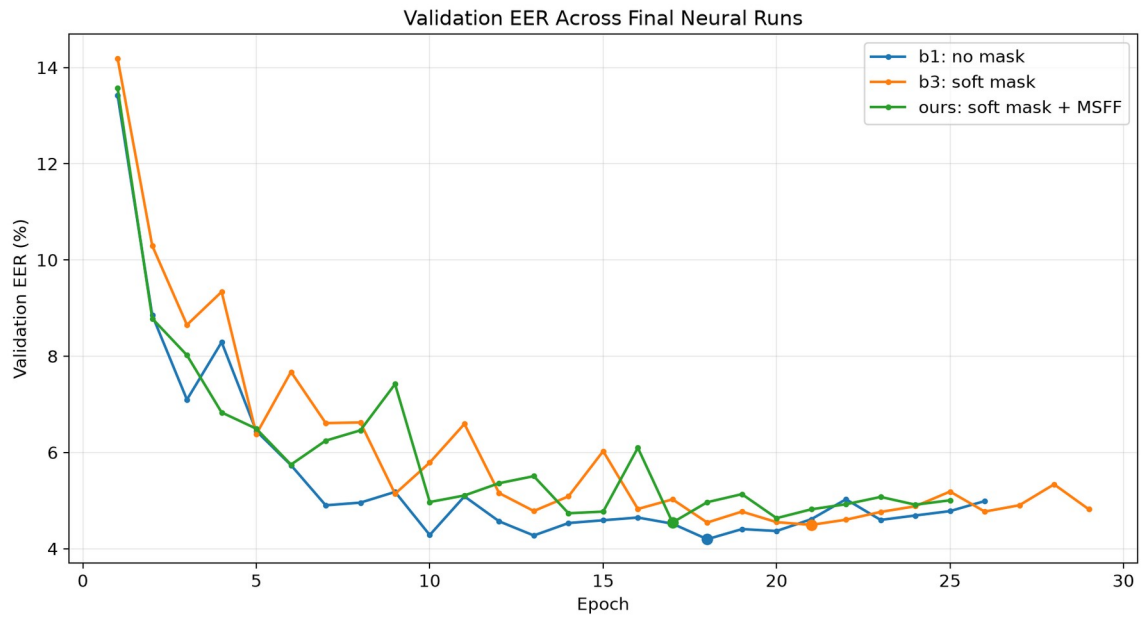


Figure 5.1. Validation EER by epoch for the final b1, b3, and MSFF neural runs; checkpoint selection used the minimum validation EER in each run.

5.5 Frozen-Test Verification Results

Quantitative verification is conducted using the frozen subject-disjoint CASIA test split and a unified pair protocol consisting of 17,508 genuine pairs and 87,540 impostor pairs. Performance metrics for the baseline configurations, the proposed model, and the classical baseline are compiled in Table 5.4. These metrics include Equal Error Rate (EER), Area Under the Curve (AUC), True Accept Rate at 0.1% False Accept Rate (TAR@0.1%FAR), and the False Accept Rate (FAR) and False Reject Rate (FRR) at the validation-selected decision threshold.

Table 5.4. Quantitative verification performance on the frozen CASIA test protocol.

Configuration	Method	Mask / fusion	EER (%)	AUC	TAR@0.1 % FAR (%)	Result status
b1_arciris_nomask	IResNet50 + ArcFace	None	7.3549	0.963291	78.8451	Final; single run
b3_arciris_softmask	IResNet50 + ArcFace	Soft angular mask	3.5915	0.991927	85.2568	Final; single run
ours_iresnet_msff_softmask	IResNet50 + ArcFace	Soft mask + MSFF	3.5498	0.992076	88.1940	Final; single run
classical_iriscode	Daugman-style log-Gabor iris code	Not applicable	8.9565	0.959427	75.2684	Final; evaluation only

The earlier notebook-derived MSFF row is not used as a final result. Table 5.5 instead records the provenance and status of the final neural and classical artifacts so that historical notebook values cannot be confused with the frozen-test comparison.

Table 5.5. Provenance and status of the results used in the final comparison.

Configuration	Frozen test protocol	Checkpoint / artifact	FAR at val. threshold	FRR at val. threshold	Status
b1_arciris_nomask	200 subjects; 17,508 genuine + 87,540 impostor pairs	Best-val-EER checkpoint; results/.../b1.../test_metrics.json	0.0788%	12.0802%	Final; single neural run
b3_arciris_softmask	Same frozen identities and pairs	Best-val-EER checkpoint; results/.../b3.../test_metrics.json	0.0400%	14.1764%	Final; single neural run
ours_iresnet_msff_softmask	Same frozen identities and pairs	Best-val-EER checkpoint; results/.../ours.../test_metrics.json	0.0640%	13.6166%	Final; single neural run
classical_iriscode	Same frozen val/test identities and score protocol	Evaluation-only; results/.../classical_iriscode.../test_metrics.json	0.1199%	24.0576%	Final; no neural training

5.6 Why a Closed-Set Confusion Matrix Is Not Used

Because this study primarily addresses open-set verification using pairwise template comparison, a traditional closed-set classification confusion matrix is not a suitable metric. Open-set verification evaluates whether two arbitrary templates belong to the same identity, whereas a confusion matrix is restricted to closed-set identification where all test subjects are known during training. Consequently, no closed-set confusion matrices are generated, preserving the scientific alignment of the evaluation protocol with open-set verification conditions.

Should closed-set identification analysis be required, a separate classification benchmark would need to be executed on a closed subset of the training identities. This separate evaluation would operate under a distinct protocol and must not be conflated with the primary open-set verification results reported in this chapter.

5.7 Model Comparison and Discussion

The no-mask b1 configuration achieves an EER of 7.3549%, an AUC of 0.963291, and a TAR@0.1% FAR of 78.8451%. The matched soft-mask b3 configuration reaches an EER of 3.5915%, while the proposed MSFF configuration reaches 3.5498%. Therefore, under this single-run comparison, the proposed configuration is numerically better than b1 by 3.8051 percentage points in EER, 0.028785 in AUC, and 9.3489 percentage points in TAR. It also improves EER by 0.0417 percentage points relative to b3. These differences are descriptive and do not establish statistical significance or causal superiority. The Daugman-

style log-Gabor reference has a higher EER of 8.9565%; it provides non-neural context and is neither an architecture ablation nor an exact OSIRIS reproduction [6].

Table 5.6. Comparative analysis of baseline, proposed, and classical models.

Comparison	Evidence	Cautious Assessment
IResNet50 no-mask (b1) vs. soft-mask (b3)	b1 EER 7.3549%; b3 EER 3.5915%	b3 is numerically better than b1 by 3.7634 percentage points in EER in this single-run comparison; this does not establish a causal or statistically significant masking effect.
IResNet50 soft-mask (b3) vs. soft-mask + MSFF (ours)	EER 3.5498% (ours) vs b3 (3.5915%)	Ours (IResNet50+MSFF+soft mask) achieves EER 3.5498%, showing a 0.0417 percentage-point improvement over matched b3.
Proposed Model vs. Baselines	Ours (3.5498%) vs b1 (7.3549%) & Daugman (8.9565%)	Ours has 3.8051 percentage points lower EER than b1 and 5.4067 points lower than the classical reference; these are descriptive single-run comparisons.
PyTorch vs. TensorRT (Jetson Nano)	FP16 latency 76.479 ms vs PyTorch 168.660 ms	TensorRT FP16 achieves a 2.21x network speedup while maintaining cosine parity.

5.8 Controlled Component Comparisons

The b1-versus-b3 comparison isolates the configured soft angular mask while retaining the same backbone, split, and score protocol. In the updated single-run results, EER decreases from 7.3549% for b1 to 3.5915% for b3, a numerical reduction of 3.7634 percentage points. The b3-versus-MSFF comparison evaluates the added fusion block under the same masking condition; EER decreases by a further 0.0417 percentage points. Possible interactions with fixed masks, angular alignment, localization error, and training stochasticity remain hypotheses; repeated seeds or subject-level uncertainty analysis are required before causal conclusions.

Table 5.7. Status of controlled architectural comparisons and component analyses.

Component	Comparative Configuration	Status
Soft angular mask	b1 vs. b3	Completed; b3 is numerically better than b1 by 3.7634 percentage points in EER in single runs.
Multi-Scale Feature Fusion	b3 vs. Ours (IResNet50+MSFF+soft-mask)	Completed; ours improves EER by 0.0417 percentage points over the matched b3 baseline.
Lightweight backbone	IResNet50 vs. MobileFaceNet (retired b4)	Retired from the active protocol after repeated failed attempts; no completed result is reported.
Edge quantization and optimization	PyTorch FP32 vs. TensorRT FP32/FP16	Completed on Jetson Nano; FP16 achieves 2.21x network speedup.

5.9 Qualitative Visualizations

Qualitative visualizations were extracted from the prototype development notebooks to interpret model behavior and feature representation. These analytical results include Receiver Operating Characteristic (ROC) curves, matching score distributions, feature space projections, and Grad-CAM++ activation maps. Since these figures are derived from historical checkpoint runs, they are intended for qualitative analysis only and do not replace quantitative test metrics.

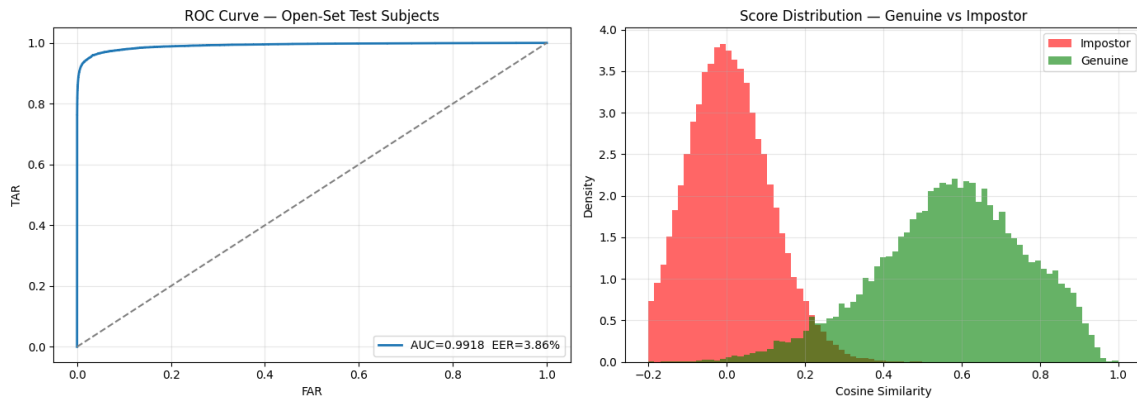


Figure 5.2. Test ROC curve and matching score distribution from the prototype notebook run.

The historical ROC and score-distribution plot in Figure 5.2 illustrates separation and overlap between genuine and impostor scores for its notebook checkpoint. Because it is not tied to the final b1, b3, or MSFF checkpoint, it is retained only as a qualitative example. Operational thresholds in the reported experiments are selected on validation data rather than from this test visualization.

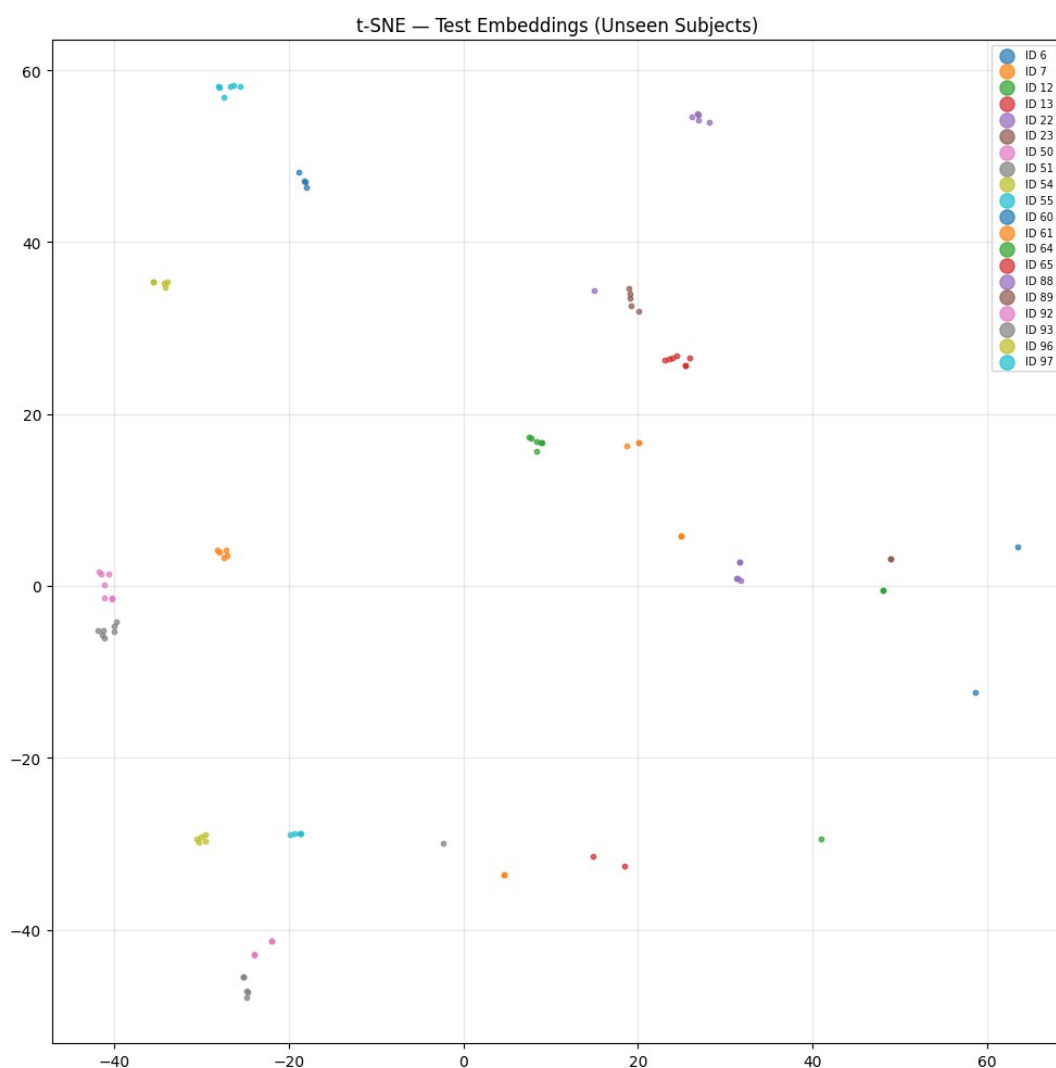


Figure 5.3. Qualitative t-SNE projection of the embedding space from the prototype notebook run.

The t-SNE projection in Figure 5.3 is a qualitative view of one historical embedding set [18]. The nonlinear reduction can distort distances and cluster geometry, so the plot cannot replace verification metrics or support quantitative claims about class separation.

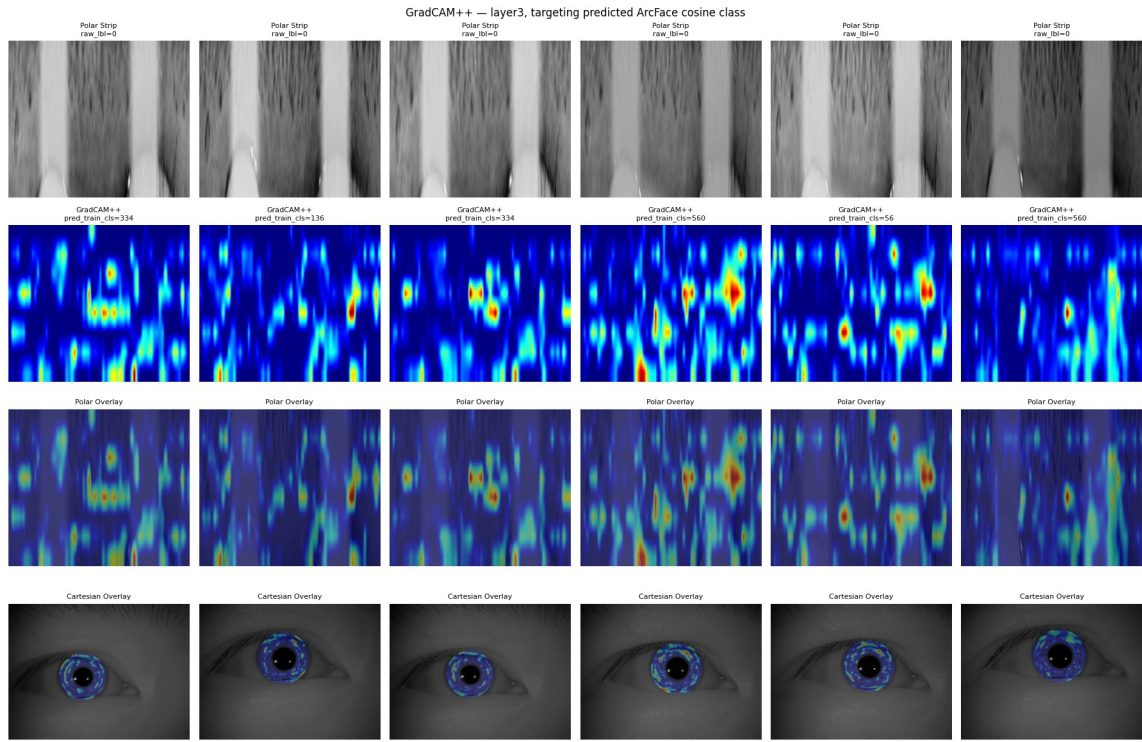


Figure 5.4. Grad-CAM++ activation heatmaps and inverse projections from the prototype notebook run.

The Grad-CAM++ heatmaps in Figure 5.4 suggest which polar and Cartesian regions influenced the historical checkpoint [17]. They may reveal attention outside the iris annulus, but they do not prove robustness, localization accuracy, or causal use of iris texture.

5.10 TensorRT and Jetson Nano Deployment Results

The final IResNet50-MSFF embedding network was exported to ONNX and compiled into TensorRT 8.2 FP32 and FP16 engines on a Jetson Nano B01 [24], [25], [38], [39]. The deployment graph contains the trained feature extractor and its learned parameters, whereas the camera controller continues to perform acquisition, localization, polar normalization, input conversion, and score calculation. This separation was intentional: it allowed the neural backend to be optimized without changing the established preprocessing and verification protocol.

Inference begins after a frame has passed the application quality gate. The controller converts the accepted eye crop to the same grayscale and 64×512 normalized polar representation used during evaluation, applies the stored input scaling, and places the result in a batch-one tensor. The selected backend then performs one forward pass and returns the feature vector. The vector is checked for finite values and non-zero norm before it is L2-normalized. For a query vector z_q and a stored template z_t , the matching score is the

cosine similarity $s = \mathbf{z}_q^T \mathbf{z}_t$. Because the vectors are normalized, a larger score indicates greater angular agreement between the query and the enrolled template.

The decision is not produced by the neural network as a class label. Instead, the application compares the score with a threshold τ selected from calibration evidence. A candidate is accepted only when $s \geq \tau$. When margin rejection is enabled, the best score must also exceed the strongest different-identity score by at least m , so the rule is $s_{\text{best}} \geq \tau$ and $s_{\text{best}} - s_{\text{second}} \geq m$. Same-side-only matching applies an additional constraint: a left-eye query is compared only with left-eye templates, and the same rule is applied to the right eye. These rules explain how the application reaches a decision while avoiding the incorrect interpretation that cosine similarity is a probability.

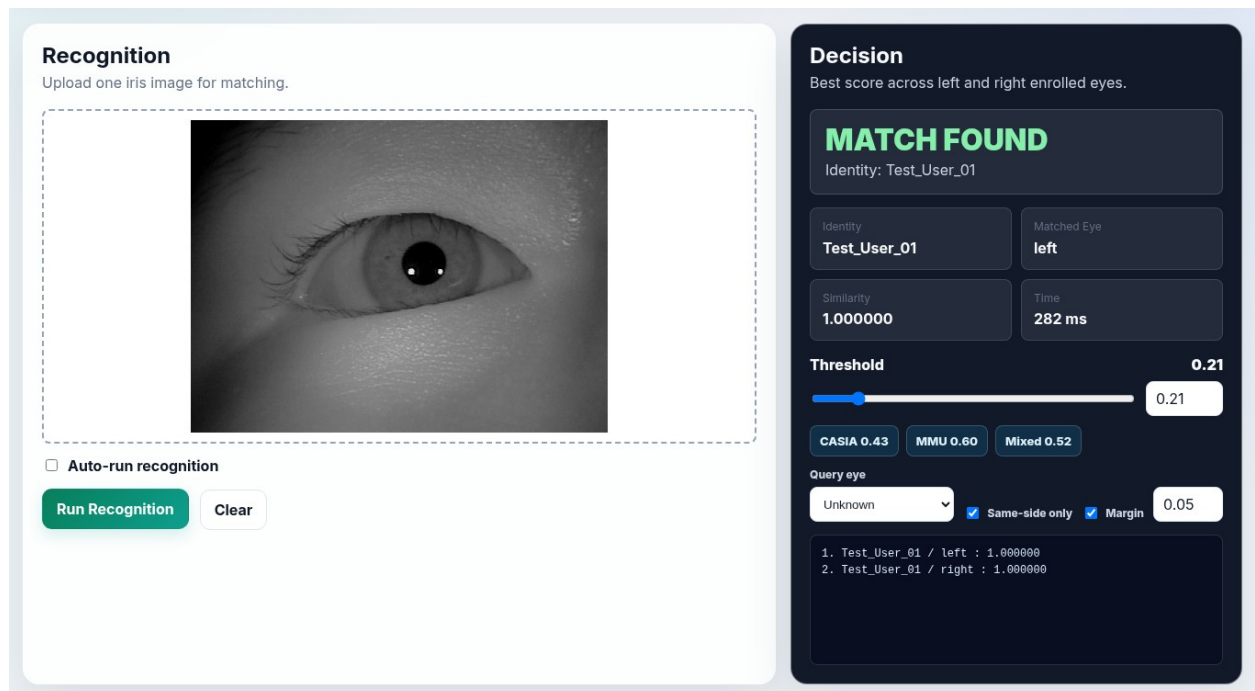


Figure 5.5. Real TensorRT FP16 recognition result produced by the deployed Jetson Nano WebUI using the final checkpoint.

Enrollment and recognition use the same backend path. During enrollment, the operator records an accepted left-eye capture and an accepted right-eye capture, the server generates one normalized embedding for each, and the local versioned database stores the identity label, eye side, and template vector. During recognition, the query follows the same preprocessing and embedding steps, the server scans the permitted templates, ranks the cosine scores, and returns the identity associated with the highest valid score together with the threshold result, competing score, eye side, and optional margin result. A “correct” live result therefore means that the score satisfied the configured operating rule for an enrolled

identity; it is not a new accuracy measurement unless the query identity and the decision outcome are recorded under a prespecified test protocol.

Enrollment / Training	
Store a person with one left-eye and one right-eye embedding.	
User name Test_User_01	
Left Eye	Right Eye
Enroll User	Clear

Enrollment Result	
Both embeddings must be valid before storage.	
ENROLLED	
User added: Test_User_01	
Left Eye	Right Eye
316 ms	189 ms
Total Time	Database
534 ms	1 users

Figure 5.6. Successful enrollment of a pseudonymous user using left- and right-eye iris embeddings.

At batch 1, PyTorch FP32 measured 168.660 ms (5.920 FPS), TensorRT FP32 measured 126.737 ms (7.875 FPS), and TensorRT FP16 measured 76.479 ms (13.035 FPS). End-to-end medians including shared preprocessing were 250.543, 208.621, and 158.362 ms, respectively. TensorRT FP16 was 2.21 $\times$ faster than PyTorch FP32, with a median cosine similarity of 0.999962 relative to the PyTorch reference. The measurements show that the practical optimization benefit came from both TensorRT graph execution and FP16 arithmetic, while the small cosine difference remained negligible for this deployment check.

The optimization procedure used the PyTorch model as the numerical baseline, exported the inference graph to ONNX, parsed and built a TensorRT FP32 engine, and then rebuilt the engine with FP16 layer execution where supported by the Nano GPU. TensorRT performs graph-level scheduling and kernel selection, while FP16 reduces arithmetic and activation-storage cost. The deployment kept batch size equal to one because the web application processes one eye crop at a time; increasing the batch would improve offline throughput but would not represent the interactive use case. Warm-up iterations were completed before timing, and execution was serialized through the backend lock so that concurrent Flask requests could not corrupt the CUDA or PyCUDA context.

The shared preprocessing median was 81.883 ms over 127 valid images, with three failures. This distinction is important for interpreting the title-level optimization claim: TensorRT accelerates the neural network, but it does not automatically remove the cost of camera

capture, JPEG validation, pupil and iris localization, Daugman normalization, or CLAHE preview generation. The measured end-to-end result therefore represents the actual operator path, whereas the network median isolates the optimization effect. Peak process memory fell from 2,790.47 MB for PyTorch FP32 to 1,274.64 MB for TensorRT FP32 and 1,126.63 MB for TensorRT FP16; the FP16 artifact was 145.50 MB compared with 170.62 MB for PyTorch FP32. INT8 was not reported because no validated calibration set and no INT8 parity artifact were produced.

The deployment-set EER values were 2.9624%, 2.9624%, and 2.9359% for PyTorch FP32, TensorRT FP32, and TensorRT FP16, respectively. These values verify conversion behavior on the 127-image deployment sample and do not replace the

Table 5.8. Performance benchmarks and precision comparison on Jetson Nano.

Backend	Network median (ms)	Throughput (FPS)	End-to-end median (ms)	Peak process RSS (MB)	Artifact size (MB)	Median cosine vs. PyTorch
PyTorch FP32	168.660	5.920	250.543	2,790.47	170.62	-
TensorRT FP32	126.737	7.875	208.621	1,274.64	290.09	+ 0%

TensorRT FP16	76.479	13.035	158.362	1,126.63	145.50	- 0.001%
------------------	--------	--------	---------	----------	--------	----------

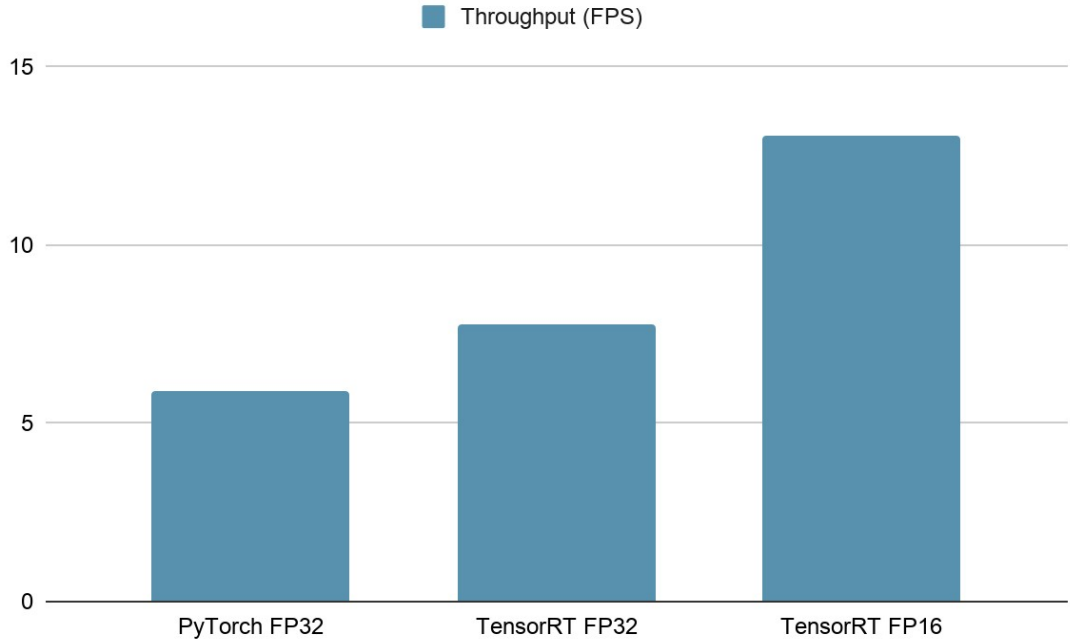


Figure 5.7: Inference throughput on the Jetson Nano at Batch 1

5.11 Threats to Validity

The neural comparisons contain one training run per configuration, so seed variability and confidence intervals are unavailable. Verification pairs also share identities and images and should not be treated as independent observations without an appropriate resampling analysis [19].

External validity is limited to the CASIA sensor domain, the recorded Jetson Nano software stack, and the stated deployment sample. The 127-image check validates conversion behavior and approximate score preservation; it is not a full frozen-CASIA TensorRT evaluation. Power telemetry, cross-sensor IMX219 accuracy, liveness, and presentation-attack resistance were not measured [22].

The web application should consequently be interpreted as an engineering demonstrator. Its quality score is a heuristic combination of detection, brightness, focus, pupil evidence, centering, eye size, and frame stability. It determines whether a frame is suitable for capture, but it has not been validated as a predictor of verification success.

CHAPTER 6. CONCLUSION AND FUTURE WORK

6.1 Main Results

This thesis established a reproducible open-set iris-verification pipeline comprising frozen subject-disjoint splitting, pupil and iris localization, Daugman normalization, ArcFace-based embedding learning, validation-based checkpoint and threshold selection, final test evaluation, and deployment of the embedding network on a Jetson Nano.

On the frozen CASIA protocol, b1 achieved 7.3549% EER, b3 achieved 3.5915%, the MSFF configuration achieved 3.5498%, and the classical log-Gabor reference achieved 8.9565%. Thus, the proposed configuration was numerically best among the neural configurations in this single-run comparison: it reduced EER by 3.8051 percentage points relative to b1 and by 0.0417 percentage points relative to b3. These differences are not claimed as statistically significant. On Jetson Nano, TensorRT FP16 reached 13.035 FPS compared with 5.920 FPS for PyTorch FP32, a $2.21\times$ speedup with median cosine parity of 0.999962. The deployment-set biometric check used 127 images and is not a full CASIA re-score.

6.2 Contributions and Learning Outcomes

The main contributions and learning outcomes are summarized as follows:

- A frozen, subject-disjoint protocol with traceable split and metric artifacts.
- A reproducible localization, normalization, embedding, and open-set verification pipeline.
- A controlled comparison of no masking, deterministic soft angular masking, and MSFF.
- An evaluation-only Daugman-style log-Gabor reference under the same validation/test identities and score protocol.
- A measured PyTorch FP32, TensorRT FP32, and TensorRT FP16 deployment comparison on Jetson Nano.
- Evidence-aware reporting that distinguishes final results, historical visualizations, and unresolved limitations.
- Practical experience in biometric evaluation, reproducible experimentation, and constrained-device deployment.

6.2.1 Technical and Research Practice

Furthermore, the project enabled the team to cultivate and enhance critical technical and practical skills:

- Python programming for image processing and deep learning.

- Constructing custom dataset classes and training pipelines in PyTorch.
- Designing robust data splits to prevent data leakage.
- Structuring source code into reusable and modular components.
- Writing specialized scripts for training, evaluation, and model export.
- Monitoring and synthesizing experimental results.
- Extracting and categorizing empirical evidence from notebooks, reports, and code repositories.
- Distinguishing actual empirical results from planned or incomplete tasks.
- Preparing technical documentation and academic reports.

6.3 Limitations

The conclusions are subject to the following limitations. The live application is an implemented deployment capability, but it is not itself evidence of cross-sensor biometric accuracy.

- The active protocol contains three completed neural configurations and one evaluation-only classical comparison. The unsuccessful b4 attempt was retired and is not reported as a result.
- Each neural configuration was trained once. Seed variability and confidence intervals are therefore unavailable, and pair scores share identities and images.
- Localization-failure statistics were not recorded consistently for every benchmark configuration.
- Presentation-attack detection and liveness were outside the scope of the experiments.
- TensorRT parity was checked on the deployment inputs, but a complete frozen-CASIA TensorRT re-score and a separate ONNX Runtime parity artifact were not preserved.
- The Jetson biometric check used 127 valid deployment images rather than the full CASIA test set. Three preprocessing failures were recorded, and input-power and energy measurements were unavailable. The accompanying calibration sample contained CASIA-style and MMU images but no IMX219 images; its thresholds are therefore diagnostic starting points, not camera-specific operating points.

The practical CASIA mirror and any derived biometric artifacts must also be reviewed against licence, privacy, and institutional release requirements before publication [21], [40].

6.4 Future Work

To address these limitations, the following future work is proposed.

- Repeat each neural configuration with multiple training seeds and report uncertainty intervals.
- Analyze localization failures, image quality, and false-match cases by split and acquisition condition.
- Re-score the complete frozen CASIA protocol with the TensorRT FP32 and FP16 engines.
- Measure power and energy together with network and end-to-end latency when a board-level measurement path is available.
- Conduct an institutionally approved IMX219 NoIR study with separate enrollment and probe sessions, anonymous participant identifiers, prespecified lighting and distance conditions, and acquisition-failure reporting [27], [28].
- Extend the demonstration with presentation-attack detection, liveness checks, and a validated camera-specific threshold before considering security-critical use.

REFERENCES

- [1] J. G. Daugman, “High Confidence Visual Recognition of Persons by a Test of Statistical Independence,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 15, no. 11, pp. 1148–1161, 1993, doi: 10.1109/34.244676.
- [2] R. P. Wildes, “Iris recognition: an emerging biometric technology,” *Proceedings of the IEEE*, vol. 85, no. 9, pp. 1348–1363, 1997, doi: 10.1109/5.628669.
- [3] J. Daugman, “How Iris Recognition Works,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 14, no. 1, pp. 21–30, Jan. 2004, doi: 10.1109/TCSVT.2003.818350.
- [4] J. Daugman, “New Methods in Iris Recognition,” *IEEE Transactions on Systems, Man and Cybernetics, Part B (Cybernetics)*, vol. 37, no. 5, pp. 1167–1175, Oct. 2007, doi: 10.1109/TSMCB.2007.903540.
- [5] K. W. Bowyer, K. Hollingsworth, and P. J. Flynn, “Image understanding for iris biometrics: A survey,” *Computer Vision and Image Understanding*, vol. 110, no. 2, pp. 281–307, May 2008, doi: 10.1016/j.cviu.2007.08.005.
- [6] N. Othman, B. Dorizzi, and S. Garcia-Salicetti, “OSIRIS: An open source iris recognition software,” *Pattern Recognition Letters*, vol. 82, pp. 124–131, Oct. 2016, doi: 10.1016/j.patrec.2015.09.002.
- [7] K. P. Hollingsworth, K. W. Bowyer, and P. J. Flynn, “The Best Bits in an Iris Code,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 31, no. 6, pp. 964–973, Jun. 2009, doi: 10.1109/TPAMI.2008.185.
- [8] Z. Zhao and A. Kumar, “Towards More Accurate Iris Recognition Using Deeply Learned Spatially Corresponding Features,” in *2017 IEEE International Conference on Computer Vision (ICCV)*, Oct. 2017, pp. 3829–3838. doi: 10.1109/ICCV.2017.411.
- [9] A. Gangwar and A. Joshi, “DeepIrisNet: Deep iris representation with applications in iris recognition and cross-sensor iris recognition,” in *2016 IEEE International Conference on Image Processing (ICIP)*, Sep. 2016, pp. 2301–2305. doi: 10.1109/ICIP.2016.7532769.
- [10] K. Nguyen, C. Fookes, A. Ross, and S. Sridharan, “Iris Recognition With Off-the-Shelf CNN Features: A Deep Learning Perspective,” *IEEE Access*, vol. 6, pp. 18848–18855, 2018, doi: 10.1109/ACCESS.2017.2784352.
- [11] K. Nguyen, H. Proença, and F. Alonso-Fernandez, “Deep Learning for Iris Recognition: A Survey,” *ACM Computing Surveys*, vol. 56, no. 9, pp. 1–35, Oct. 2024, doi: 10.1145/3651306.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, “Identity Mappings in Deep Residual Networks,” in *Lecture Notes in Computer Science*, 2016, pp. 630–645. doi: 10.1007/978-3-319-46493-0_38.
- [13] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2019, pp. 4685–4694. doi: 10.1109/CVPR.2019.00482.
- [14] R. Alinia Lat, S. Danishvar, H. Heravi, and M. Danishvar, “Boosting Iris Recognition by Margin-Based Loss Functions,” *Algorithms*, vol. 15, no. 4, p. 118, Mar. 2022, doi: 10.3390/a15040118.

- [15] S. K. Khan, P. J. Flynn, and A. Czajka, “Lowering the Barrier to IREX Participation: Open-Source Algorithms, Toolkit, and Benchmarking for Iris Recognition,” arXiv:2605.20735, 2026. [Online]. Available: <https://arxiv.org/abs/2605.20735>
- [16] S. Lei, A. Shan, B. Liu, Y. Zhao, and W. Xiang, “Lightweight and efficient dual-path fusion network for iris segmentation,” *Scientific Reports*, vol. 13, no. 1, Aug. 2023, doi: 10.1038/s41598-023-39743-w.
- [17] A. Chattopadhyay, A. Sarkar, P. Howlader, and V. N. Balasubramanian, “Grad-CAM++: Generalized Gradient-Based Visual Explanations for Deep Convolutional Networks,” in 2018 IEEE Winter Conference on Applications of Computer Vision (WACV), Mar. 2018, pp. 839–847. doi: 10.1109/WACV.2018.00097.
- [18] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008. [Online]. Available: <https://www.jmlr.org/papers/v9/vandermaaten08a.html>
- [19] J. C. Wu, A. F. Martin, and R. N. Kacker, “Validation of Nonparametric Two-sample Bootstrap in ROC Analysis on Large Datasets,” *Communications in Statistics - Simulation and Computation*, vol. 45, no. 5, pp. 1689–1703, May 2016, doi: 10.1080/03610918.2015.1065327.
- [20] D. P. Benalcazar, J. E. Tapia, M. Vasquez, L. Causa, E. L. Droguett, and C. Busch, “Toward an Efficient Iris Recognition System on Embedded Devices,” *IEEE Access*, vol. 11, pp. 133577–133590, 2023, doi: 10.1109/ACCESS.2023.3337033.
- [21] Center for Biometrics and Security Research, Chinese Academy of Sciences, “CASIA-IrisV4,” 2020. [Online]. Available: <https://hycasia.github.io/dataset/casia-irisv4/> (accessed Jul. 12, 2026).
- [22] ISO/IEC, “Information technology—Biometric performance testing and reporting—Part 1: Principles and framework,” ISO/IEC 19795-1:2021, 2021. [Online]. Available: <https://www.iso.org/standard/73515.html> (accessed Jul. 12, 2026).
- [23] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature Pyramid Networks for Object Detection,” in 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Jul. 2017, pp. 936–944. doi: 10.1109/CVPR.2017.106.
- [24] NVIDIA Corporation, “NVIDIA TensorRT Documentation.” [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/latest/index.html> (accessed Jul. 12, 2026).
- [25] NVIDIA Corporation, “NVIDIA Jetson Nano System-on-Module Data Sheet,” 2019. [Online]. Available: <https://developer.nvidia.com/embedded/dlc/jetson-nano-system-module-data-sheet> (accessed Jul. 12, 2026).
- [26] ONNX Project, “Introduction to ONNX.” [Online]. Available: <https://onnx.ai/onnx/intro/> (accessed Jul. 12, 2026).
- [27] Raspberry Pi Ltd, “Camera Module 2.” [Online]. Available: <https://www.raspberrypi.com/products/camera-module-v2/> (accessed Jul. 12, 2026).
- [28] International Electrotechnical Commission, “Photobiological safety of lamps and lamp systems,” IEC 62471:2006, 2006. [Online]. Available: <https://webstore.iec.ch/en/publication/7076> (accessed Jul. 12, 2026).

- [29] T.-T. Huynh, D.-T. Huynh, C.-S. Duong, H.-S. Nguyen, Q. H. Nguyen, and L.-T. Tu, “Efficient Iris Recognition via Polar Representation and Radial Stripe Attention,” *Computer Modeling in Engineering & Sciences*, vol. 147, no. 2, pp. 1–10, 2026, doi: 10.32604/cmes.2026.080616.
- [30] Y. Mi et al., “ImmerIris: A Large-Scale Dataset and Benchmark for Off-Axis and Unconstrained Iris Recognition,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2026. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2026/papers/Mi_ImmerIris_A_Large-Scale_Dataset_and_Benchmark_for_Off-Axis_and_Unconstrained_CVPR_2026_paper.pdf
- [31] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.
- [32] R. O. Duda and P. E. Hart, “Use of the Hough transformation to detect lines and curves in pictures,” *Communications of the ACM*, vol. 15, no. 1, pp. 11–15, Jan. 1972, doi: 10.1145/361237.361242.
- [33] D. J. Field, “Relations between the statistics of natural images and the response properties of cortical cells,” *Journal of the Optical Society of America A*, vol. 4, no. 12, p. 2379, Dec. 1987, doi: 10.1364/JOSAA.4.002379.
- [34] Y.-H. Li and M. Savvides, “An Automatic Iris Occlusion Estimation Method Based on High-Dimensional Density Estimation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 4, pp. 784–796, Apr. 2013, doi: 10.1109/TPAMI.2012.169.
- [35] G. W. Quinn, P. Grother, and J. Matey, “IREX IX Part One: Performance of Iris Recognition Algorithms,” *National Institute of Standards and Technology, NISTIR 8207*, 2018, doi: 10.6028/NIST.IR.8207.
- [36] G. W. Quinn, J. Matey, E. Tabassi, and P. Grother, “IREX V: Guidance for Iris Image Collection,” *National Institute of Standards and Technology, NISTIR 8013*, 2014, doi: 10.6028/NIST.IR.8013.
- [37] Organization of Scientific Area Committees for Forensic Science, “Standard Guide for Capturing Iris Images for Use with Iris Recognition Systems,” *OSAC 2024-N-0004*, 2024. [Online]. Available: <https://www.nist.gov/system/files/documents/2024/02/05/OSAC%202024-N-0004%20Standard%20Guide%20for%20Capturing%20Iris%20Images%20for%20Use%20with%20Iris%20Recognition%20Systems%20Version%201.0.pdf>
- [38] NVIDIA Corporation, “JetPack SDK 4.6.1,” 2022. [Online]. Available: <https://developer.nvidia.com/embedded/jetpack-sdk-461> (accessed Jul. 12, 2026).
- [39] NVIDIA Corporation, “NVIDIA TensorRT 8.2.5 Developer Guide,” 2022. [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/archives/tensorrt-825/pdf/TensorRT-Developer-Guide.pdf> (accessed Jul. 12, 2026).
- [40] DeepInsight, “InsightFace: State-of-the-art 2D and 3D face analysis project,” *GitHub*. [Online]. Available: <https://github.com/deepinsight/insightface> (accessed Jul. 19, 2026).